Can a pulsar timing array phase itself up?

A single self-contained demo, end to end. One invented population of supermassive black-hole binaries; the gravitational-wave background it makes; what real arrays can see of it; and whether measuring pulsar distances from those sources can feed back and improve the array.

The five things this notebook shows

- **V1** — the population we invented, and the background it adds up to.
- **V2** — what four different arrays can actually detect in that background.
- **V3** — why measuring one pulsar's distance is hard, even with several sources.
- **V4** — the wheel: detect → recover → certify → deepen → repeat, and where it stops.
- **V4b** — what you get once the distances are certified and the array is phased up.
- **V5** — why taking the tallest peak fails, and what certification does instead.
- **V6** — what it buys for the background measurement.

How to read the labels. Every number in this notebook is one of four kinds:

tag	meaning
REAL	computed by real code we did not write (hasasia sensitivity curves)
BANKED	measured by our own campaigns; must not be tuned here
DEMO	a knob invented for this demo to have the right shape; carries no scale
ILLUSTRATIVE	invented purely to draw a picture; nothing depends on it

PROVISIONAL marks anything downstream of `combs* = 7.8`, which is under a null-test audit. Those numbers are lower bounds, not settled values.

The honest summary, in one table

what	status	the short version
sensitivity curves	**REAL**	hasasia `DeterSensitivityCurve`. The background inside each curve is the fitted amplitude of the drawn population — self-consistent, no separate knob.
background amplitude	**REAL target**	normalised to NANOGrav 15 yr, A = 6.4×10^{-15} (90% CI 3.7–10.6). This draw realises 6.2×10^{-15}.
the population itself	**DEMO**	a toy. Argue with the knobs in V1; the machinery does not care.
chirp-mass slope −3.6	**BANKED-derived**	fixed by the audited bright-end counts slope γ_eff = 1.56, not chosen.
conversion 0.5, combs* 7.8	**BANKED**	V28. combs* is **PROVISIONAL**.
depth gain 0.05–0.19 dex	**BANKED**	measured range; we use the top of it.

what	status	the short version
GWB sharpening 20–40%	**BANKED**	V26.
field baseline 92–93% wrong	**BANKED**	what uncertified practice achieves.
detection bar SNR ≥ 3	**DEMO, declared**	standard forecasting bar, but *below* the conventional 5–8. See V2.
baseline 30 yr, 26 TOA/yr	**DEMO**	a strong lever. See V2.
timing precision 100 ns	**matches VERNIER-24**	`SIG_TOA = 100e-9`, the median residual RMS our own campaign uses. Earlier drafts ran 30 ns, which was picked arbitrarily and inflated K by 25–30%.
F_BEND = 2 nHz turnover	**DEMO**	where the background stops following f^(−2/3). See V1.
comb-usability scale (4.0)	**DEMO, unjustified**	the weakest number here. It decides whether the wheel turns at all. See V4.
jitter cap 0.4 rad, depth scaling, plot subsampling, log display bins	**DEMO**	shape only.
distance prior 10 pc, uniform	**DEMO, optimistic**	`PRIOR_PC` from V24 (Xiao et al., SKA-era projection). The real catalogue has median σ_d = 242 pc and only 3 of 86 pulsars below 10 pc. See V3.
pulsar distances, source direction	**ILLUSTRATIVE**	V4b picture only.

Three things this notebook gets wrong, stated up front

- **It is CW-optimistic.** The mass function is a pure power law with no Schechter cutoff, so 23% of the background comes from the top mass decade alone, and the two loudest drawn sources sit *above* the real NANOGrav CW upper limit. **Every K here is an upper bound.**
- **It is under-occupied by ~270×.** The real sky holds ~4.6×10^4 binaries per frequency bin at 10 nHz; this draw holds ~170. Matching it needs 10^8 sources, which is not drawable in a notebook.
- **The counts slope comes out at 1.9, not the audited 1.56.** The hard mass ceiling and hard distance horizon push the brightest end back toward Euclidean.

In [1]:

```
import numpy as np
import matplotlib.pyplot as plt
import hasasia.sim as hsim
import hasasia.sensitivity as hsen
plt.rcParams.update({"figure.dpi": 110, "font.size": 11,
                     "axes.spines.top": False, "axes.spines.right": False})

G_SI, C_SI = 6.674e-11, 2.998e8
MSUN, MPC  = 1.989e30, 3.086e22
YR_S = 3.15576e7
C_PC_S = C_SI / 3.0856775814913673e16
T_YR  = 30                  # observing baseline [yr] -- the strongest lever here
T_OBS = T_YR * YR_S
F_YR  = 1 / YR_S

# banked anchors
CONVERSION, COMBS_STAR, DEPTH_DEX_MAX = 0.5, 7.8, 0.19
PRIOR_PC    = 10.0        # distance prior combs* was MEASURED at [V24 PRIOR_PC, Xiao et al.]
```

```
NATS_PER_COMB = 1.055    # evidence per comb [V28 fit: S = 1.055*combs - 3.42]   # V28 / V28(PROVISIONAL) / banked range top
```

V1 — The population we invented

What the plots show

- *Left:* every binary we drew (frequency vs strain, coloured by mass), with the background they add up to in red, and the fitted f^(−2/3) line in black.
- *Right:* the mass function we drew from.

How it works

- We draw M_TOT binaries, each with a frequency, a chirp mass and a distance.
- Each one has a strain h. We add them up per frequency bin to get the background.
- We then fit a single amplitude A to that sum. **That is the only background in this notebook** — the curves in V2 carry this same A. No background is injected separately.

What we assumed, and why

- **Frequency law** — binaries pile up at low frequency because they linger there. For GW-driven inspiral the residence time goes as f^(−8/3), which is what we use above the bend. [DEMO]
- **F_BEND = 2 nHz** — below this the law goes flat. Real binaries at wide separation are driven by their *environment* (stars scattering off them, gas discs), not by GWs, so they move through low frequencies faster and pile up less. Literature puts this bend anywhere from 0.1 to 10 nHz; 2 nHz is mid-range. **The turnover you see is this knob, not a discovery.** [DEMO]
- **Mass slope −3.6** — *derived, not chosen.* In a bounded volume the bright-end source counts go as h^(−0.6(γ_m−1)), and our audit measured that exponent as 1.56, which fixes γ_m = 3.6. The old −1.5 would mean a sky owned by one screaming binary. [BANKED-derived]
- **Masses 3×10^{8}–10^{10} M⊙** — the lower end is the black-hole mass-function knee; cutting there costs 4% of the background. The upper end is the largest black hole ever observed.
- **Distances 30–1500 Mpc, uniform in volume**, no evolution.
- **M_TOT = 340,000** — *not free.* It is whatever makes the background land on the NANOGrav amplitude.

The two vertical dotted lines. Red at **1/T = 1.06 nHz** — the lowest frequency a 30-year baseline can resolve; nothing to the left of it is measurable, and it is also where the red curve switches binning. Green at **F_BEND = 2 nHz** — the turnover. Note the bend sits *inside* the resolvable band, which is why the amplitude fit is restricted to f ≥

F_BEND: fitting a pure f^(−2/3) across the turnover drags A low. A 1/T bin is 1.06 nHz *wide*, so a single one would swallow the whole sub-nHz decade. Below ~1 pc spacing the comb is finer than the screen — the banding there is moiré, not structure.

The realism check is λ_fore: the loudest source's share of its own frequency bin. Our audit measured 0.31 on the real sky. The old "top 10% of all sources" number is still printed but is a different statistic and must not be read against 0.31.

In [2]:

```
ALPHA_F   = -8/3            # GW-driven residence time, ABOVE the bend
F_RANGE   = (2e-10, 1e-7)
F_BEND    = 2e-9            # environmental-hardening turnover (DEMO knob, stated)
MC_SLOPE  = -3.6            # chirp-mass exponent  <-- DERIVED from banked gamma_eff =
1.56
MC_RANGE  = (3e8, 1e10)     # M_sun: BH mass-function knee -> largest observed BH
D_RANGE   = (30, 1500)      # Mpc, uniform-in-volume
M_TOT     = 340_000         # pinned by A_gwb = NANOGrav 15 yr
A_NG15    = 6.4e-15         # NANOGrav 15 yr, gamma = 13/3 (90% CI 3.7-10.6e-15)
LAM_AUDIT = 0.31            # banked audited lambda_fore [V24]
N_AUD10   = 6.9e4*(20/T_YR) # audited occupancy at 10 nHz [V24], rescaled to our 1/T bins
PLOT_SUB  = 20_000          # dots drawn per panel (plot styling only)

rpop = np.random.default_rng(3)
g = ALPHA_F
# dN/dlnf ~ max(f, F_BEND)^ALPHA_F : GW-driven above the bend, flat below it
W_FLAT = F_BEND**g * np.log(F_BEND/F_RANGE[0])
W_GW   = (-1/g) * (F_BEND**g - F_RANGE[1]**g)
u = rpop.uniform(0, 1, M_TOT) * (W_FLAT + W_GW)
f_src = np.where(u < W_FLAT,
                 F_RANGE[0]*np.exp(np.minimum(u, W_FLAT)/F_BEND**g),
                 (F_BEND**g + g*np.maximum(u - W_FLAT, 0))**(1/g))
e = MC_SLOPE + 1
mc_src = (MC_RANGE[0]**e + rpop.uniform(0, 1, M_TOT)*(MC_RANGE[1]**e - MC_RANGE[0]**e))**
(1/e) * MSUN
d_src  = (D_RANGE[0]**3 + rpop.uniform(0, 1, M_TOT)*(D_RANGE[1]**3 - D_RANGE[0]**3))**
(1/3) * MPC
h_src  = (8/np.sqrt(10)) * (G_SI*mc_src)**(5/3) * (np.pi*f_src)**(2/3) / (C_SI**4 * d_src)

# realized background from THIS draw
bin_edges = np.arange(F_RANGE[0], F_RANGE[1], 1/T_OBS)
hc2_bin = np.zeros(len(bin_edges)-1)
bidx = np.digitize(f_src, bin_edges) - 1
ok = (bidx >= 0) & (bidx < len(hc2_bin))
np.add.at(hc2_bin, bidx[ok], (h_src**2 * f_src * T_OBS)[ok])
hmax_bin = np.zeros(len(hc2_bin))                    # loudest single source per bin
np.maximum.at(hmax_bin, bidx[ok], (h_src**2 * f_src * T_OBS)[ok])
n_per_bin = np.bincount(bidx[ok], minlength=len(hc2_bin))
bin_cent = 0.5*(bin_edges[1:] + bin_edges[:-1])
# fit the -2/3 law only where the population actually follows it: resolvable (f >= 1/T)
# AND above the bend. At T = 30 yr, 1/T = 1.06 nHz sits BELOW F_BEND, so the naive
# band would straddle the turnover and bias A low.
mfit = (hc2_bin > 0) & (bin_cent < 1e-8) & (bin_cent >= max(1/T_OBS, F_BEND))
A_GWB = np.exp(np.mean(np.log(np.sqrt(hc2_bin[mfit])) +
(2/3)*np.log(bin_cent[mfit]/F_YR)))

lam_fore = float(np.median(hmax_bin[mfit]/hc2_bin[mfit]))
lam_top = np.sort(h_src**2*f_src)[-int(0.1*M_TOT):].sum() / (h_src**2*f_src).sum()
i10 = int(np.argmin(np.abs(bin_cent - 1e-8)))
print(f"Realized background: A_gwb = {A_GWB:.2e} at f_yr (fitted to the draw itself)")
print(f"  target: NANOGrav 15 yr A = {A_NG15:.1e}  ->  ratio {A_GWB/A_NG15:.2f}")
print(f"lambda_fore = {lam_fore:.2f}  (loudest source's share of its OWN bin, median over
"
      f"the {mfit.sum()} fit bins)")
print(f"  banked audited sky: lambda_fore = {LAM_AUDIT}  [V24]  -- like-for-like")
print(f"Share of background power in the brightest 10% of ALL sources: {lam_top:.2f}")
print(f"  NOT comparable to lambda_fore (different denominator); kept as the toy's own "
```

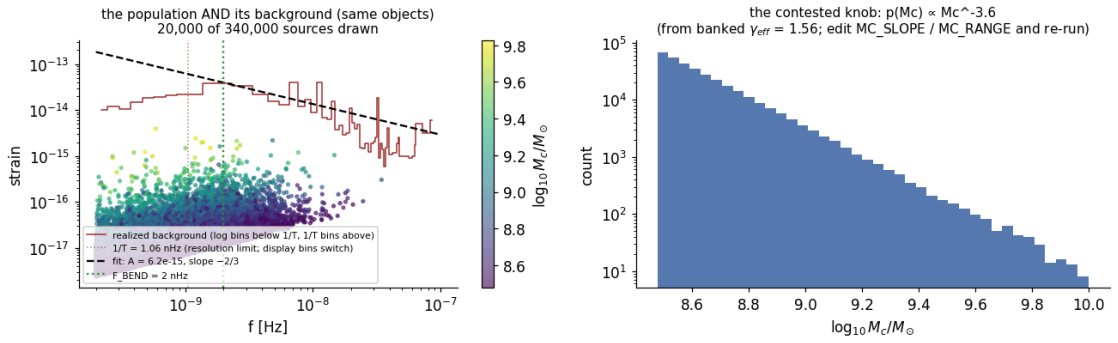
```
      f"top-heaviness, and it is still severe")
print(f"Occupancy: {n_per_bin[i10]} sources in the 10 nHz bin (audited sky: {N_AUD10:.1e}
for "
      f"{T_YR}-yr bins) -- the toy is ~{N_AUD10/max(n_per_bin[i10],1):.0f}x under-
occupied; "
      f"see the honesty box")

# Below 1/T the PTA has NO frequency resolution, so the physical 1/T bins lump the
# whole sub-bend decade into a single point (bin 0 alone holds ~90% of the draw).
# h_c^2 = sum h^2 f/df is already a per-log-f quantity, so re-binning that region in
# log f carries the same meaning at usable resolution. DISPLAY ONLY: the A fit and
# lambda_fore above use the physical 1/T bins untouched.
disp_edges = np.logspace(np.log10(F_RANGE[0]), np.log10(1/T_OBS), 9)
disp_cent  = np.sqrt(disp_edges[1:]*disp_edges[:-1])
disp_hc2   = np.zeros(len(disp_cent))
didx = np.digitize(f_src, disp_edges) - 1
dok  = (didx >= 0) & (didx < len(disp_hc2))
np.add.at(disp_hc2, didx[dok], (h_src**2 * f_src)[dok])
disp_hc2 /= np.diff(disp_edges)
_mphys = (hc2_bin > 0) & (bin_cent >= 1/T_OBS)          # physical bins, no double-count
bg_f  = np.concatenate([disp_cent, bin_cent[_mphys]])
bg_hc = np.concatenate([np.sqrt(disp_hc2), np.sqrt(hc2_bin[_mphys])])

isub = np.random.default_rng(99).choice(M_TOT, PLOT_SUB, replace=False)
fig, axes = plt.subplots(1, 2, figsize=(12, 4.2))
ax = axes[0]
sc = ax.scatter(f_src[isub], h_src[isub], c=np.log10(mc_src[isub]/MSUN), s=6,
cmap="viridis",
                alpha=0.6, rasterized=True)
ax.plot(bg_f, bg_hc, color="#a03030", lw=1, drawstyle="steps-mid",
        label="realized background (log bins below 1/T, 1/T bins above)")
ax.axvline(1/T_OBS, color="#a03030", ls=":", lw=1, alpha=0.6,
           label=f"1/T = {1e9/T_OBS:.2f} nHz (resolution limit; display bins switch)")
ffit = np.logspace(np.log10(F_RANGE[0]), -7, 50)
ax.plot(ffit, A_GWB*(ffit/F_YR)**(-2/3), "k--", lw=1.5, label=f"fit: A = {A_GWB:.1e},
slope -2/3")
ax.axvline(F_BEND, color="#3a8f3a", ls=":", lw=1.5, label=f"F_BEND = {F_BEND*1e9:.0f}
nHz")
ax.set_xscale("log"); ax.set_yscale("log")
ax.set_xlabel("f [Hz]"); ax.set_ylabel("strain")
ax.set_title(f"the population AND its background (same objects)\n"
             f"{PLOT_SUB:,} of {M_TOT:,} sources drawn", fontsize=10)
ax.legend(fontsize=7, loc="lower left")
plt.colorbar(sc, ax=ax, label=r"$\log_{10} M_c/M_\odot$")
ax = axes[1]
ax.hist(np.log10(mc_src/MSUN), bins=40, color="#2b5aa0", alpha=0.8)
ax.set_yscale("log")
ax.set_xlabel(r"$\log_{10} M_c/M_\odot$"); ax.set_ylabel("count")
ax.set_title(f"the contested knob: p(Mc) ∝ Mc^{MC_SLOPE}\n"
             f"(from banked $\\gamma_{{eff}}$ = 1.56; edit MC_SLOPE / MC_RANGE and re-
run)",
             fontsize=10)
fig.suptitle(f"V1 — explicit, editable SMBHB toy population, normalised to NANOGrav 15 yr;
"
             f"the background is its sum  ($\\lambda_{{fore}}$ = {lam_fore:.2f} vs audited
"
             f"{LAM_AUDIT})", y=1.05)
plt.tight_layout(); plt.savefig("fig_v1_population.png", bbox_inches="tight"); plt.show()
```

```
Realized background: A_gwb = 6.24e-15 at f_yr (fitted to the draw itself)
  target: NANOGrav 15 yr A = 6.4e-15  ->  ratio 0.98
lambda_fore = 0.28  (loudest source's share of its OWN bin, median over the 7 fit bins)
  banked audited sky: lambda_fore = 0.31  [V24]  -- like-for-like
Share of background power in the brightest 10% of ALL sources: 0.98
  NOT comparable to lambda_fore (different denominator); kept as the toy's own top-
heaviness, and it is still severe
Occupancy: 170 sources in the 10 nHz bin (audited sky: 4.6e+04 for 30-yr bins) -- the toy is
~271x under-occupied; see the honesty box
```

V1 — explicit, editable SMBHB toy population, normalised to NANOGrav 15 yr; the background is its sum (λ_{fore} = 0.28 vs audited 0.31)



V2 — What each array can actually see

What the plot shows

- Four arrays looking at the *same* sky. Grey dots are the population, blue dots are the sources that array can detect.
- Grey dashed line: the raw hasasia curve — the confusion wall made by the background.
- Black line: the detection bar we actually apply.

How detection is decided — and why the obvious version is wrong

- hasasia gives a curve h_c and an SNR formula. Putting them together, **SNR = (h / h_c) × √(fT)**.
- So "is the source above the curve?" is **not** an SNR = 1 test. It is a *sliding* bar — about 1 at f = 1/T, rising to about 7 at 100 nHz. It has no single meaning.
- We instead fix **SNR ≥ 3** and plot the rescaled curve, so "above the black line" means the same thing everywhere. [DEMO, declared]
- SNR ≥ 3 is a standard forecasting bar but is **below** the conventional detection bar of 5–8. The cell prints what happens at SNR ≥ 1 and ≥ 8 so you can see how much rests on this choice.

Pulsar count is the main lever — but timing precision is not free

- **Below ~10 nHz the band is confusion-limited**: the noise is the background itself, so the curves for 30 ns and 100 ns are identical there (within 1%). More pulsars helps; better clocks do not.
- **Above ~10 nHz that breaks down.** The 100 ns curve is 1.4× higher at 30 nHz and 3.1× at 100 nHz — white noise, not background. A good fraction of the detectable sources live there, so going 30 ns → 100 ns costs **25–30% of K** and moves the ignition threshold a whole rung, from 80 pulsars to 100.
- Every rung sits at **100 ns**, which is what VERNIER-24 uses as the median residual RMS. An earlier draft ran 30 ns; that was an arbitrary choice and it was flattering the result. At fixed timing K climbs **9 → 13 → 16 → 24 → 28 → 36** across 60 → 200 pulsars, so the ladder is still a pulsar-count ladder — just an honest one. It is

still climbing at 200, roughly 0.17 sources per pulsar: no sign yet of saturating against the confusion floor.

Why a 30-year baseline

- Observing time is the strongest lever here: K grows roughly *linearly* with it. At 15 years the same threshold needed ~160 pulsars; at 30 years it needs ~80.
- The gain is capped by F_BEND — the extra low frequencies a long baseline opens up land in the flat part of the population, not the pile-up. [DEMO]

In [3]:

```
freqs = np.logspace(np.log10(F_RANGE[0]), -7, 100)
SNR_BAR = 3.0        # DECLARED detection bar (see the markdown above)

def make_curve(n_psr, sigma_ns, seed=42):
    r = np.random.default_rng(seed)
    phi = r.uniform(0, 2*np.pi, n_psr); theta = np.arccos(r.uniform(-1, 1, n_psr))
    psrs = hsim.sim_pta(timespan=T_YR, cad=26, sigma=sigma_ns*1e-9, phi=phi, theta=theta,
                        A_gwb=A_GWB, freqs=freqs)
    hc = hsen.DeterSensitivityCurve([hsen.Spectrum(p, freqs=freqs) for p in psrs]).h_c
    return hc, phi, theta

def det_curve(hc, bar=None):
    """hasasia h_c rescaled to a FIXED SNR bar.

    hasasia: h_c = sqrt(f*S_eff) and SNR = h*sqrt(T/S_eff)  [Hazboun+ 2019, Eq. 79]
      =>  SNR = (h/h_c)*sqrt(f*T)  =>  SNR >= bar  <=>  h >= bar*h_c/sqrt(f*T).
    Comparing h to the RAW h_c is thus a SLIDING bar (~1 at 1/T, ~7 at 100 nHz), not
    an SNR = 1 cut. Deepening the curve by d dex raises SNR by d dex, so the wheel's
    0.19 dex gain carries over unchanged.
    """
    return (SNR_BAR if bar is None else bar) * hc / np.sqrt(freqs*T_OBS)

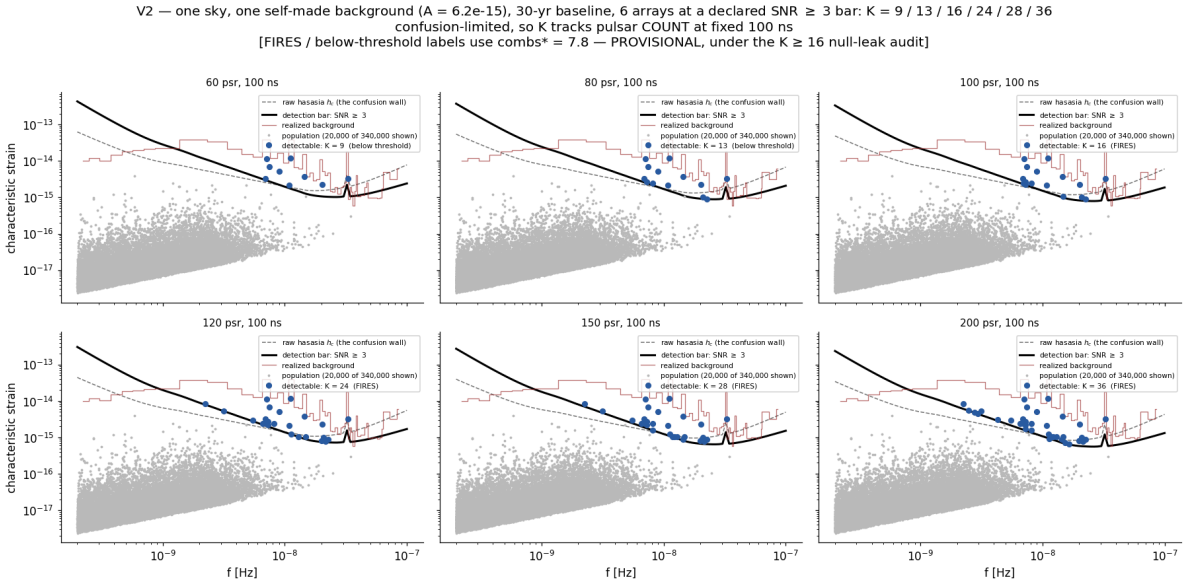
ERAS = [(60, 100), (80, 100), (100, 100), (120, 100), (150, 100), (200, 100)]
hc_raw, hc_era, geo_era = {}, {}, {}
for _n, _s in ERAS:
    hc_raw[(_n, _s)], _phi, _th = make_curve(_n, _s)
    hc_era[(_n, _s)] = det_curve(hc_raw[(_n, _s)])
    geo_era[(_n, _s)] = (_phi, _th)

WORK = (100, 100)                       # ignition-era array: V3, V5, V6 all run on this
one
hc_work = hc_era[WORK]
phi_p, theta_p = geo_era[WORK]
N_PSR = WORK[0]

def detected(hc): return h_src > np.interp(f_src, freqs, hc)

fig, axes = plt.subplots(2, 3, figsize=(16.5, 8.0), sharex=True, sharey=True)
for ax, (n, s) in zip(axes.ravel(), ERAS):
    hc = hc_era[(n, s)]; det = detected(hc)
    fires = "FIRES" if np.floor(CONVERSION*det.sum()) >= COMBS_STAR else "below threshold"
    ax.loglog(freqs, hc_raw[(n, s)], color="#777", lw=1, ls="--",
              label="raw hasasia $h_c$ (the confusion wall)")
    ax.loglog(freqs, hc, "k", lw=2, label=f"detection bar: SNR $\\geq$ {SNR_BAR:.0f}")
    ax.loglog(bg_f, bg_hc, color="#a03030", lw=0.8, drawstyle="steps-mid", alpha=0.7,
              label="realized background")
    m = ~det[isub]
    ax.loglog(f_src[isub][m], h_src[isub][m], ".", color="#bbb", ms=3,
              label=f"population ({PLOT_SUB:,} of {M_TOT:,} shown)")
    ax.loglog(f_src[det], h_src[det], "o", color="#2b5aa0", ms=5,
              label=f"detectable: K = {det.sum()}  ({fires})")
    ax.set_title(f"{n} psr, {s} ns", fontsize=10)
    ax.legend(fontsize=7, loc="upper right")
for ax in axes[1]:
    ax.set_xlabel("f [Hz]")
```

```
for ax in axes[:, 0]:
    ax.set_ylabel("characteristic strain")
fig.suptitle("V2 — one sky, one self-made background (A = %.1e), %d-yr baseline, %d arrays
"
             "at a declared SNR $\\geq$ %.0f bar: K = %s\nconfusion-limited, so K tracks
pulsar "
             "COUNT at fixed 100 ns\n[FIRES / below-threshold labels use combs* = 7.8 — "
             "PROVISIONAL, under the K ≥ 16 null-leak audit]"
             % (A_GWB, T_YR, len(ERAS), SNR_BAR,
                " / ".join(str(int(detected(hc_era[k]).sum())) for k in ERAS)), y=1.01)
plt.tight_layout(); plt.savefig("fig_v2_curves.png", bbox_inches="tight"); plt.show()
print(f"baseline: T = {T_YR} yr, cadence 26/yr;  detection bar: SNR >= {SNR_BAR:.0f}
(hasasia Eq. 79)")
for n, s in ERAS:
    K = int(detected(hc_era[(n, s)]).sum())
    print(f"  {n:3d} psr / {s:3d} ns : K = {K:3d}  ->  K_rec =
{int(np.floor(CONVERSION*K)):3d} "
          f"{'>=' if np.floor(CONVERSION*K) >= COMBS_STAR else '< '} combs* =
{COMBS_STAR}")
print(f"\nBRACKET — the same {len(ERAS)} arrays, same sky, under other detection bars:")
for _B in (1.0, 3.0, 8.0):
    _ks = [int(detected(det_curve(hc_raw[k], _B)).sum()) for k in ERAS]
    _nf = sum(1 for _K in _ks if np.floor(CONVERSION*_K) >= COMBS_STAR)
    _tag = "  <-- optimistic" if _B == 1 else ("  <-- conventional" if _B == 8 else "  <--
in force")
    print(f"  SNR >= {_B:.0f}: K = {' / '.join(f'{k:3d}' for k in _ks)}"
          f"   -> {_nf} of {len(ERAS)} arrays ignite{_tag}")
```




```
baseline: T = 30 yr, cadence 26/yr;  detection bar: SNR >= 3 (hasasia Eq. 79)
   60 psr / 100 ns : K =   9  ->  K_rec =   4 <  combs* = 7.8
   80 psr / 100 ns : K =  13  ->  K_rec =   6 <  combs* = 7.8
  100 psr / 100 ns : K =  16  ->  K_rec =   8 >= combs* = 7.8
  120 psr / 100 ns : K =  24  ->  K_rec =  12 >= combs* = 7.8
  150 psr / 100 ns : K =  28  ->  K_rec =  14 >= combs* = 7.8
  200 psr / 100 ns : K =  36  ->  K_rec =  18 >= combs* = 7.8

BRACKET — the same 6 arrays, same sky, under other detection bars:
  SNR >= 1: K =  82 / 103 / 125 / 143 / 168 / 218   -> 6 of 6 arrays ignite  <-- optimistic
  SNR >= 3: K =   9 /  13 /  16 /  24 /  28 /  36   -> 4 of 6 arrays ignite  <-- in force
  SNR >= 8: K =   3 /   5 /   6 /   6 /   6 /   7   -> 0 of 6 arrays ignite  <--
conventional
```

V3 — Why one pulsar distance is hard

What the plots show

- *Top row:* the likelihood surface for three pulsars, in nats. This is where the argument lives.
- *Bottom row:* the same thing as a posterior after a 10 pc distance prior.
- **The dashed gold line is the GATE FLOOR, and it is the whole decision.** It is measured for *that pulsar* by scrambling *its own* combs (see V5). The rule is simply:

  > **certify only if exactly ONE peak is above the line.**

  The winner sits at 0 by construction, so "one peak above" means every rival has been pushed below what chance alignment can fake. Orange markers are the peaks still above the line — the ones blocking certification — and the count and verdict are in each title.

- **The floor rises with your own comb strength.** Stronger combs make bigger chance alignments, so they raise the bar as well as the signal. Pulsar 1 has the strongest combs and therefore the *highest* floor — 50 nats at N_CW = 4, against 20 and 27 for the others. Being decisive in raw contrast is not the same as certifying.
- **What the frames show.** At N_CW = 4 **all three are refused** (3, 4 and 4 peaks still above their floors). By N_CW = 12 all three certify, with floors of 87, 65 and 64 nats and a single peak above each.
- **Do not read the panels as a yield estimate.** They were *chosen* at N_CW = 4 to span the outcomes, so they are not a random sample at any other N_CW. V5's measured yields are the number to quote.
- Green = the true distance. Red = the tallest peak.

How it works

- Each detected source gives a *comb*: the pulsar's distance enters the signal phase periodically, so one source tells you the distance only up to a repeating spacing.
- Different sources have different spacings. Enough overlapping combs leave only one distance consistent with all of them — a vernier scale.
- **N_CW = how many combs you have in hand.**

Why the top row and not the bottom

- The likelihood swings hundreds of nats. On a linear posterior axis everything except the tallest spike vanishes, and an undetermined problem *looks* solved. The nats axis shows every rival.

The three pulsars are chosen, not drawn

- V5 measures that **~46%** of geometries lock onto a *wrong* tooth at N_CW = 4. Three arbitrary draws would probably all look clean and would contradict V5.
- So we screen 60 candidate geometries and show one of each outcome:

The three labels below describe the *likelihood* outcome at N_CW = 4, not the gate's verdict — a panel can be labelled "decisive" and still be refused.

label	what it means
RIGHT tooth, decisive	peak on the true distance, nearest rival far below it — genuinely determined
RIGHT tooth, but a rival ties	peak on the true distance, but a rival sits 0.2 nats away — *right by luck*, 5 teeth still live
WRONG tooth	peak on a different distance entirely — the panel title gives the offset

- **The middle one is the point of the notebook.** The answer is correct and worthless, because nothing in the surface tells you it is correct.

Two things to watch as you step N_CW

- With **3 combs or fewer the surface is exactly degenerate** — identical repeat teeth, 58 to 103 of them live. This is our banked "you need at least 3" rule appearing on its own, out of the geometry.
- **By 8 combs all three collapse to a single live tooth** — which is `combs* = 7.8`. The dial and the banked threshold agree.

Assumptions

- **GWB phase jitter** — the unresolved background shifts each comb's phase a little, capped at 0.4 rad. [DEMO]
- **10 pc distance prior, the same for every pulsar** — and this is the most optimistic number in the notebook. It is `PRIOR_PC` from VERNIER-24 (Xiao et al.), an **SKA-era projection**, not a present-day value. Our own canonical catalogue (`CW_transition/best_distances.txt`, 86 pulsars) has **median σ_d = 242 pc**, with only **3 below 10 pc and none below 1 pc** — so the notebook gives every pulsar a prior better than 83 of 86 real ones, and 24× tighter than the median. [DEMO, optimistic]
- It also **cannot be uniform in reality**: $\sigma_d = \sigma_\pi \cdot d^2$, so distance error grows as the square of distance and spans orders of magnitude across an array. The banked VLBI result is that only ~6 pulsars reach sub-pc and J0437 alone carries 93 % of certification power. Certification would concentrate in a few nearby pulsars rather than spreading evenly.
- And it is **centred on the truth** here, which flatters it further — a real parallax prior sits on a noisy measurement.

Does it change the answer? Mostly not: the prior enters V5 only through `q_max`, and the measured floor is what binds (the `% clearing` column matches the gate yield exactly). But it does make the *search* easier. The prior sets how many candidate fringes must be told apart — the census measure is $K = 6\sigma/\Delta$, so ~120 candidates at 10 pc against ~2900 at the real median. The ±25 pc grid here is ±2.5σ of a 10 pc

prior; a realistic prior needs a ~30× wider search, which raises the trials floor. The Anchor Census found **no pulsar reaches K ≤ 1 even with real published priors** — only J0437 gets to K ≤ 3 — and concluded *"fringe identification is the binding constraint."*

In [4]:

```
det_idx = np.where(detected(hc_work))[0]
loud = det_idx[np.argsort(h_src[det_idx])[::-1]]          # loudness-ranked detectable
sources
K_MAX = len(loud)
hcbg_at = np.interp(f_src[loud], bin_cent, np.sqrt(np.maximum(hc2_bin, 1e-40)))
snr_src = h_src[loud] / np.interp(f_src[loud], freqs, hc_work)   # loudness in units of
                                                                 # the SNR>=3 detection
bar
N_DUMMY, N_SEL, N_CAND = 3, 4, 60
dL = np.linspace(-25, 25, 30000)

def make_geometry(rng):
    mu  = np.arccos(rng.uniform(-0.9, 0.9, K_MAX))
    kap = 2*np.pi * f_src[loud] * (1 - np.cos(mu)) / C_PC_S
    rho = np.clip(4.0 * snr_src * rng.uniform(0.3, 1.0, K_MAX), 0, 12)
    sigphi = np.clip(hcbg_at / h_src[loud], 0.05, 0.4)   # GWB phase jitter, capped 0.4
rad (DEMO knob)
    return kap, rho, rng.normal(0, sigphi)               # frozen: the slider stays stable

def lnL(geo, n_cw):
    """Log-likelihood surface over the distance offset, in nats (unnormalised)."""
    kap, rho, eps = geo
    if n_cw == 0:
        return np.zeros_like(dL)
    return -np.sum((rho[:n_cw, None]**2/2) *
                   (1 - np.cos(kap[:n_cw, None]*dL[None, :] - eps[:n_cw, None])), axis=0)

def tooth_gap(pen):
    """argmax offset [pc], and the best WRONG tooth relative to the peak [nats]."""
    return dL[np.argmax(pen)], pen[np.abs(dL) > 0.15].max() - pen.max()

# The three exhibits are CHOSEN to span the real spread of outcomes rather than taken from
# an arbitrary seed. V5 measures ~43% of geometries locking a wrong tooth at N_CW = 4, so
# three lucky draws would misrepresent the problem -- and would contradict V5.
rsel = np.random.default_rng(17)
scored = [(g,) + tooth_gap(lnL(g, N_SEL)) for g in (make_geometry(rsel) for _ in
range(N_CAND))]
wrong_c = [s for s in scored if abs(s[1]) > 0.25]
right_c = sorted([s for s in scored if abs(s[1]) <= 0.25], key=lambda s: s[2])
psr_geo   = [right_c[0][0], right_c[-1][0], wrong_c[0][0]]
PSR_LABEL = ["RIGHT tooth, decisive",          # peak on truth, runner-up far below
             "RIGHT tooth, but a rival ties",  # peak on truth, runner-up ~0 nats away
             f"WRONG tooth — locked {abs(wrong_c[0][1]):.1f} pc off"]   # computed, not
fixed
print(f"exhibit screening at N_CW = {N_SEL}: {len(wrong_c)}/{N_CAND} "
      f"({100*len(wrong_c)/N_CAND:.0f}%) of these geometries lock a WRONG tooth. V5
measures the "
      f"same quantity on 300 fresh draws -- see its N_CW = {N_SEL} row. The three panels
below "
      f"are one of each outcome, chosen at N_CW = {N_SEL}, NOT a random sample.")

def binned_env(x, y, nbins=2200, fill=-np.inf):
    """Upper envelope of y on a coarse grid -- teeth are far finer than a screen pixel."""
    edges = np.linspace(x.min(), x.max(), nbins+1)
    idx = np.clip(np.digitize(x, edges)-1, 0, nbins-1)
    ymax = np.full(nbins, fill); np.maximum.at(ymax, idx, y)
    return 0.5*(edges[1:]+edges[:-1]), ymax

TOOTH_PC, N_SCR_DISP = 0.25, 80     # tooth window [pc]; scrambles per panel

def contrast_of(pen):
```

```
    """The gate's statistic: best peak minus best rival at least TOOTH_PC away from it."""
    best = dL[np.argmax(pen)]
    return pen.max() - pen[np.abs(dL - best) > TOOTH_PC].max()

def gate_floor(geo, n_cw, rng):
    """PERMUTATION floor for THIS pulsar's own combs (same construction as V5).

    Scramble these combs' phases so no true distance exists, and see how big a lead chance
    alignment manufactures. Distribution-free, so it survives the fact that our likelihood
    does not model the GWB phase jitter. Conditional on this pulsar's own rho -- a floor
    pooled over all pulsars runs 30-50% too high.
    """
    kap, rho, _ = geo
    n = min(n_cw, len(kap))
    if n == 0:
        return np.inf
    return np.percentile([contrast_of(-np.sum((rho[:n, None]**2/2) *
                          (1 - np.cos(kap[:n, None]*dL[None, :] - e[:, None])), axis=0))
                          for e in rng.uniform(0, 2*np.pi, (N_SCR_DISP, n))], 99.75) + 5.0

def rival_teeth(xb, yb, cut):
    """Positions and heights of distinct teeth rising above the gate line at -cut."""
    idx = np.where(yb > -cut)[0]
    if idx.size == 0:
        return np.array([]), np.array([])
    groups = np.split(idx, np.where(np.diff(idx) > 1)[0] + 1)
    top = np.array([g[np.argmax(yb[g])] for g in groups])
    return xb[top], yb[top]

def dial_figure(N_CW, save=None):
    fig, axes = plt.subplots(2, N_DUMMY, figsize=(12.5, 5.8), sharex=True)
    for j in range(N_DUMMY):
        pen = lnL(psr_geo[j], N_CW)
        rel = pen - pen.max()
        off, gap = tooth_gap(pen) if N_CW else (0.0, 0.0)

        ax = axes[0, j]                                   # --- LIKELIHOOD surface, in
nats
        xb, yb = binned_env(dL, rel)
        floor_j = gate_floor(psr_geo[j], N_CW, np.random.default_rng(500+j))
        rx, ry = rival_teeth(xb, yb, floor_j); nriv = len(rx)
        lnl_lo = -max(45.0, floor_j*1.25)
        ax.fill_between(xb, lnl_lo, np.maximum(yb, lnl_lo), color="#2b5aa0", lw=0)
        # anything still above the gate line is a candidate you cannot rule out
        ax.fill_between(xb, -floor_j, np.maximum(yb, -floor_j),
                        where=yb > -floor_j, color="#e07b39", lw=0, zorder=1)
        ax.plot(rx, ry + abs(lnl_lo)*0.035, "v", color="#e07b39", ms=5, zorder=4,
                clip_on=False, label="peak above the gate line" if j == 0 else None)
        ax.axvline(off if N_CW else 0, color="#a03030", ls="-", lw=1.1, label="argmax",
                   zorder=2)
        ax.axvline(0, color="#3a8f3a", ls="--", lw=1.4, label="truth", zorder=3)
        ax.axhline(-floor_j, color="#e8b84b", lw=1.6, ls="--",
                   label="GATE FLOOR — certify only if ONE peak is above it"
                   if j == 0 else None)
        ax.set_ylim(lnl_lo, abs(lnl_lo)*0.12)
        _v = "CERTIFIES" if nriv == 1 else "refused"
        ax.set_title(f"pulsar {j+1} — {PSR_LABEL[j]}\nargmax {off:+.2f} pc;  floor "
                     f"{floor_j:.0f} nats;  {nriv} peak{'' if nriv == 1 else 's'} above it
"
                     f"→ {_v}", fontsize=8.5)
        if j == 0:
            ax.set_ylabel("ln L − max  [nats]")
            ax.legend(fontsize=6, loc="lower left", framealpha=0.92)

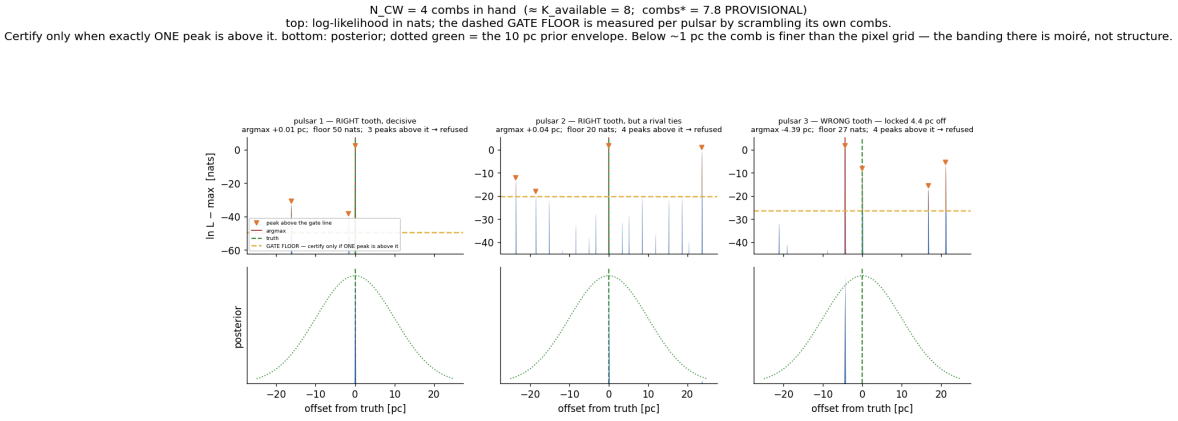
        ax = axes[1, j]                                   # --- posterior (prior applied)
        post = np.exp(pen - pen.max() - 0.5*(dL/PRIOR_PC)**2)   # parallax-grade prior
        # coarser bins here on purpose: past N_CW ~ 4 the posterior is a single needle and
        # a 1-pixel spike reads as an empty panel
        xb, pb = binned_env(dL, post/post.max(), nbins=420, fill=0.0)
        ax.axvline(0, color="#3a8f3a", ls="--", lw=1.4, zorder=0)
        ax.fill_between(xb, pb, color="#2b5aa0", lw=0, zorder=2)
```

```
        ax.plot(xb, np.exp(-0.5*(xb/PRIOR_PC)**2), color="#3a8f3a", ls=":", lw=1.2,
zorder=3)
        ax.set_ylim(0, 1.08)
        ax.set_xlabel("offset from truth [pc]"); ax.set_yticks([])
        if j == 0:
            ax.set_ylabel("posterior")
    fig.suptitle(f"N_CW = {N_CW} combs in hand  (≈ K_available =
{int(np.ceil(N_CW/CONVERSION))};"
                 f"  combs* = {COMBS_STAR} PROVISIONAL)\n"
                 f"top: log-likelihood in nats; the dashed GATE FLOOR is measured per
pulsar by "
                 f"scrambling its own combs.\nCertify only when exactly ONE peak is above
it. "
                 f"bottom: posterior; dotted green "
                 f"= the 10 pc prior envelope. Below ~1 pc the comb is finer than the
pixel "
                 f"grid — the banding there is moiré, not structure.", y=1.15)
    plt.tight_layout()
    if save: plt.savefig(save, bbox_inches="tight")
    plt.show()

try:        # live slider (works in a running Jupyter kernel)
    from ipywidgets import interact, IntSlider
    interact(lambda N_CW: dial_figure(N_CW), N_CW=IntSlider(4, 0, K_MAX, 1))
except Exception as err:
    print("ipywidgets unavailable here — use the static grid below.", err)
```

exhibit screening at N_CW = 4: 20/60 (33%) of these geometries lock a WRONG tooth. V5 measures the same quantity on 300 fresh draws -- see its N_CW = 4 row. The three panels below are one of each outcome, chosen at N_CW = 4, NOT a random sample.

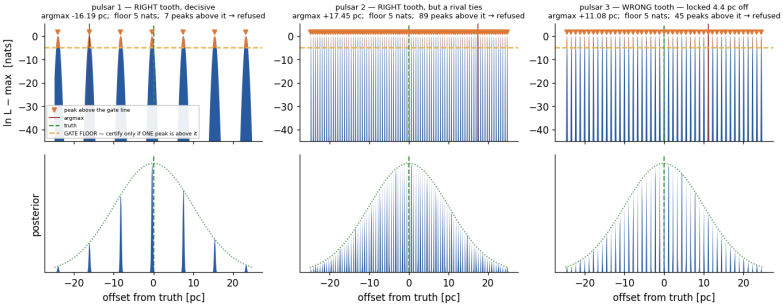
N_CW 4



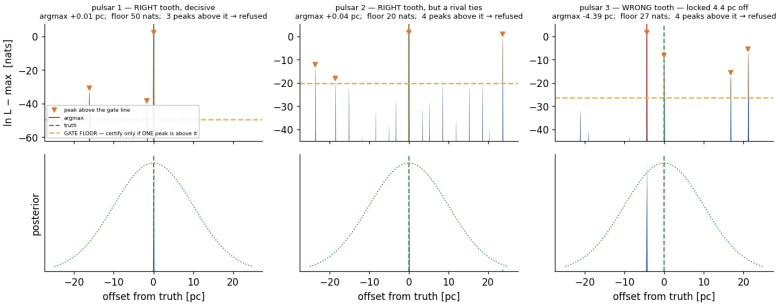
In [5]:

```
DIAL_FRAMES = (1, 4, 8, 12, K_MAX)
for n in DIAL_FRAMES:
    dial_figure(n, save=f"fig_v3_dial_N{n:02d}.png")
```

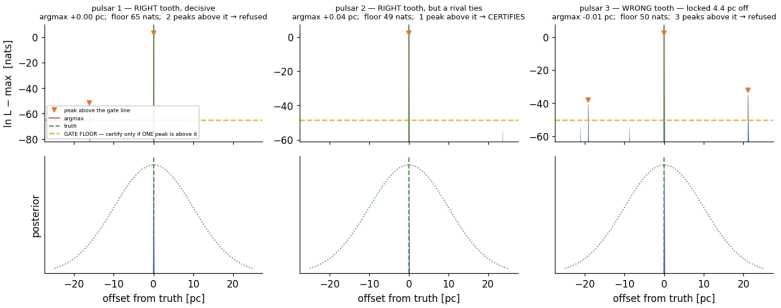
N_CW = 1 combs in hand (≈ K_available = 2; combs* = 7.8 PROVISIONAL)
top: log-likelihood in nats; the dashed GATE FLOOR is measured per pulsar by scrambling its own combs.
Certify only when exactly ONE peak is above it. bottom: posterior; dotted green = the 10 pc prior envelope. Below ~1 pc the comb is finer than the pixel grid — the banding there is moiré, not structure.



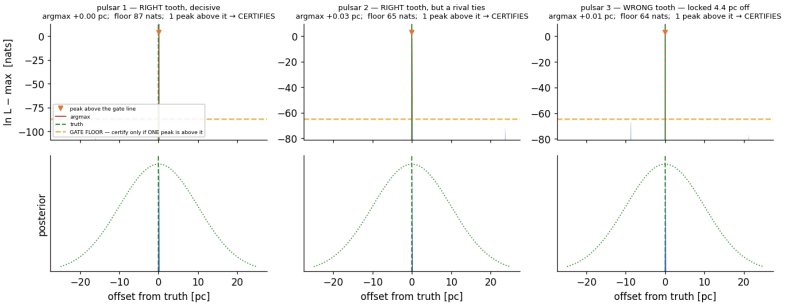
N_CW = 4 combs in hand (≈ K_available = 8; combs* = 7.8 PROVISIONAL)
top: log-likelihood in nats; the dashed GATE FLOOR is measured per pulsar by scrambling its own combs.
Certify only when exactly ONE peak is above it. bottom: posterior; dotted green = the 10 pc prior envelope. Below ~1 pc the comb is finer than the pixel grid — the banding there is moiré, not structure.



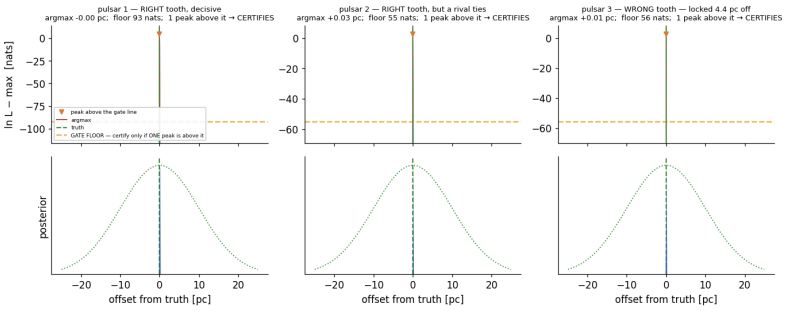
N_CW = 8 combs in hand (≈ K_available = 16; combs* = 7.8 PROVISIONAL)
top: log-likelihood in nats; the dashed GATE FLOOR is measured per pulsar by scrambling its own combs.
Certify only when exactly ONE peak is above it. bottom: posterior; dotted green = the 10 pc prior envelope. Below ~1 pc the comb is finer than the pixel grid — the banding there is moiré, not structure.



N_CW = 12 combs in hand (≈ K_available = 24; combs* = 7.8 PROVISIONAL)
top: log-likelihood in nats; the dashed GATE FLOOR is measured per pulsar by scrambling its own combs.
Certify only when exactly ONE peak is above it. bottom: posterior; dotted green = the 10 pc prior envelope. Below ~1 pc the comb is finer than the pixel grid — the banding there is moiré, not structure.



N_CW = 16 combs in hand (≈ K_available = 32; combs* = 7.8 PROVISIONAL)
top: log-likelihood in nats; the dashed GATE FLOOR is measured per pulsar by scrambling its own combs.
Certify only when exactly ONE peak is above it. bottom: posterior; dotted green = the 10 pc prior envelope. Below ~1 pc the comb is finer than the pixel grid — the banding there is moiré, not structure.



V4 — The wheel

What the plot shows

- Each panel is one array. Each coloured line is one turn of the loop; darker → lighter as it turns. Dots are the sources newly reached on that turn.
- The 60-pulsar panel has two identical lines — it never starts.

The loop, in plain terms

1. **Detect** — K sources clear the SNR ≥ 3 bar.
2. **Recover** — you only get usable waveforms for about half of them. `rec = 0.5 × K`. These are your combs. [BANKED, V28] — see "why not all of them?" below.
3. **Certify** — each pulsar checks how many of those combs are strong enough *in its own data*. If at least 7.8 are, that pulsar's distance is certified. [BANKED, PROVISIONAL]
4. **Deepen** — the certified fraction makes the array coherent, which lowers the curve by up to 0.19 dex. [BANKED range]
5. **Repeat** — a lower curve reaches more sources.

The three numbers, which are easy to confuse

	counts	set by
K	*sources* the array can detect	sky + array
rec	*combs* you actually recovered = 0.5 × K	BANKED conversion
certified	*fraction of your pulsars* with a pinned-down distance	BANKED combs* = 7.8

The first two count sources. The third counts pulsars.

Why isn't rec the same as K? Detection and recovery ask different questions.

- **Detection** asks *"is there excess power at this frequency?"* That is what the SNR ≥ 3 bar buys. It tells you a source exists.
- **A comb** asks *"where exactly is it on the sky?"* The comb spacing is κ = 2πf(1 − cos μ)/c, and **μ is the angle between the pulsar and the source**. Get the sky

> position wrong and κ is wrong, so the teeth land in the wrong places. A misplaced comb does not merely fail to help — it plants a wrong fringe.

How accurate does the sky position have to be? Teeth are ~0.5 pc apart and a 10 pc prior spans ~20 of them, so κ must be right to ~1 part in 40 — for typical geometry, the source located to **a fraction of a degree**. The banked numbers say that is the hard part: the blind F-statistic sky floor is **~6°**, and deliverable R's break-even sky prior is **0.19° per loud source** — the true fringe only wins if you are pre-localised **32× tighter than blind**. Detecting at 6° is comfortable; localising to 0.2° is not.

Which sources are crowded — measured on this sky. The 16 detectable sources are spread over 94 bins, but they are *not* spread evenly, because the population piles up at low frequency (dN/dln f ∝ f^−8/3):

f [nHz]	sources in that bin	detectables in it	this source's share of its own bin
6.9–7.4	571	**6**	0.56, 0.22, 0.05, 0.03, 0.03, 0.02
9.0	268	1	0.78
11.1	114	1	0.94
20.1	13	1	0.84
32.8	4	1	**0.99**

At the top of the band a detection **owns its bin** — 4 sources present, one detectable, holding 99 % of the power. Easy to localise. At the bottom, one bin holds 571 sources with *six* detectable among them, and the brightest owns only 56 % while others hold 2–5 % each. Those are minority shareholders in a crowded bin, competing with five other resolvable sources and 565 faint ones.

Counting the detectable sources that hold more than half of their own bin gives **8 of 16 — the banked 0.5**. Suggestive, but treat it as a coincidence of scale rather than a derivation: the 50 % cut is arbitrary, this is one sky, and V28's own mechanism is subtler.

What V28 actually found, including one counterintuitive part. More *detectable* sources in a bin **helps**: conversion rises 0.43 → 0.47 → 0.56 as K_available goes 6 → 9.5 → 18, because *"a bin holding two detectables gives the fitter a second peak to lock on rather than a blend to sit between"*. And splits are **0.00 in every cell** — the fitter never mistakes one source for two. Its failure mode is *"components landing on nothing available"*.

So resolvable sources are not competing with each other. **What costs you is the faint, unresolvable sea at the same frequency** — the thing that leaves the fit sitting between sources rather than on one. That is why the low-frequency bins above, with 571 sources apiece, are the hostile ones: not because six detectables share the bin, but because 565 unresolvable ones do.

Why localisation is hard: the bin is crowded. A 1/T bin is 1.06 nHz wide and holds ~170 sources in this toy — about 4.6×10^4 in the real sky. They radiate at nearly the same frequency and the fit sees their *sum*. The loud one dominates the amplitude, but the rest add a random-phase pedestal that pulls its apparent position off. V28 puts it as *"the other ~290 in-bin sources ARE signal"*, and measures the consequence: realized per-bin separability is **2–3, not the 5–10 Fisher predicts**.

So `rec = 0.5 × K` is a localisation cut, not a detection cut. Sky precision improves roughly as 1/SNR, so the louder half of your detections reach the fraction-of-a-degree they need and the marginal half do not. That is also why the notebook takes the **loudest** 0.5 × K — loudness stands in for "localised well enough to be a ruler".

What V28 measured, for the record. Running the search at K_available = 6.0 it *claims* 12.5 components, of which **2.58 are genuine** (2.58/6.0 = 0.43); 8.50 land on the faint sea, 1.42 are clean phantoms, and 0.00 are splits. At K_available = 18.0 it is 10.08/18.0 = 0.56. Hence the banked ≈0.5. Conversion *rises* with K_available (0.43 → 0.47 → 0.56), which the notebook's flat 0.5 does not capture — using the measured trend would make the wheel turn slightly harder, so the flat value is the conservative choice.

Why the first turn is only partial. At the 100-pulsar array the first turn has 8 combs, and "at least 7.8" means at least 8 — so a pulsar certifies only if *all 8* work for it. 80% manage that. By the next turn there are 18 combs and any 8 will do, so everyone clears.

How much the wheel is worth: about ×2.2 in K, and that number is derivable. A curve 0.19 dex deeper reaches 10^(0.19 γ_eff) more sources, and this population's realized bright-end counts slope is γ_eff = 1.9, giving **10^0.36 ≈ 2.3×**. Measured across the eras that fire in one turn: **2.25× (120 psr), 2.14× (150), 1.97× (200)**. It does **not** grow with the array — a bigger array starts higher and finishes higher, but the multiplier is the same, because the depth cap is the same. The 100-pulsar era looks better (16 → 44, 2.75×) only because it is the one era whose first turn is *partial* (80%), so it takes two productive turns instead of one.

What actually changes from turn to turn: the depth of the curve. Everything else follows from it. For the 100-pulsar era the curve sits at 0 dex (baseline, K=16), then 0.152 dex deeper (80 % certified × 0.19), then 0.190 dex deeper (100 % certified), and then stops moving. K follows the curve down: 16 → 36 → 44.

The repeated last row in each era is the stall check, not a failed turn. The loop breaks when K stops changing, so it always runs one extra iteration to *confirm* that. That is why every era ends on a duplicate — and why the last two curves in each panel lie exactly on top of one another. The 60-pulsar era is all stall check: nothing certifies, so the curve never moves and K never changes.

Why it stalls. The curve is always `original × 10^(−0.19 × certified)` — recomputed from the original each turn, never compounding. So the total possible gain is capped

at 0.19 dex. Once certification hits 100% the curve is at its floor and K stops moving. **The wheel converges to a fixed point; it does not bootstrap.**

Does it use partial information? No — and that is deliberate. The wheel is all-or-nothing at three points: per comb (ρ = 1.9 counts for nothing, ρ = 2.1 counts fully), per pulsar (7 usable combs contributes what 0 does), and per source (only the loudest `rec = 0.5K` are used at all).

That is not a simplification we should soften, because **the campaign has already tested softening it, twice, and both are NO-GOs:**

- **Deliverable R** asked whether the honestly fringe-marginalised posterior concentrates at truth. It *smears*: f = Z_needle/(Z_needle+Z_plateau) = 6.9×10^{-7} — the needle is 28 nat higher at its peak but loses 14 nat on sky volume to the wrong-fringe plateau. The verdict is recorded as **information-theoretic, not search-only**, and was computed under favourable assumptions. Break-even needs the sky pre-localised 32× tighter than the blind F-statistic floor — i.e. an EM counterpart.
- **VERNIER-28 R3** put a marginalised pulsar block into a real search: conversion **0.416 ± 0.024 against the control's 0.503 ± 0.025, Δ = −0.087 ± 0.021, excluding zero.** With distances marginalised the pulsar block cannot separate "signal here" from "source here", and the extra components are phantoms *bought with signal, not noise*.

There is also a physical reason the hard cut is right rather than merely safe: **coherent phase-up needs the distance known to a fraction of a GW wavelength.** A pulsar 80% of the way to certification does not deliver 80% of the coherent gain — an unresolved phase adds incoherently, not partially.

What the near-misses do tell us is a sensitivity, not an understatement. At the ignition turn (8 combs in hand, so certifying needs all 8) 80 pulsars certify, **19 sit at exactly 7 and 1 at 6**. So moving combs* from 7.8 to 6.8 would take the first turn from 80 % to 99 %. combs* is **PROVISIONAL**, from a fit crossing whose own bar drifts 4.40–5.28 across cells — so the first turn's certified fraction is fragile to a constant that is still under audit. The *threshold* is not: at 80 pulsars only 6 combs exist, so even perfect geometry falls short of the bar however it moves within its uncertainty.

What ρ is, and what it is not. `ρ = 4.0 × (h/h_c) × geometry` is the signal-to-noise of one source's **pulsar term in one pulsar's data** — it sets how deep that comb's fringes are (each comb contributes $\rho^2/2$ nats). A comb counts for that pulsar if ρ > 2 [VERNIER-1 bar].

Note this is a *per-pulsar* quantity while the detection bar in V2 is an *array* quantity: the Earth term adds coherently across pulsars, the pulsar term cannot, because its phase is what carries each pulsar's distance. The two are related by the array's heterogeneity, and the 4.0 that connects them **has no derivation here** — it is a demo knob. [DEMO]

But it is not the binding constraint, and that is measured. VERNIER-24: *"41 % of Arm B's pulsar-term power sits in pulsars above VERNIER-1's ρ_P ≥ 2 bar, and zero pulsars certify. The bar was never the binding constraint here."* Its diagnosis is that *"the currency is K, the number of simultaneously detectable sources at distinct frequencies."* This notebook reproduces that: at the ignition turn the per-pulsar comb counts are 6, 7 and 8 **out of the 8 available**, so the ρ bar is nearly saturated and what binds is the number of combs.

The same source gives a second independent check: V24's *"a single source cannot certify at any SNR"* is exactly V3's measured N_CW ≤ 3 exact degeneracy, where the best-wrong-tooth gap is 0.0 nats at **any** comb strength.

The prior was never absent — it was hidden inside combs*

`combs* = 7.8` is not a constant of nature. V24 certifies against an explicit trials factor **lnK = ln(6σ_prior/dL_min)** and searches **±3σ of the prior**, so a wider prior means more candidate fringes to tell apart. Evidence accrues at 1.055 nats per comb [V28], so

> **combs*(σ) = 7.8 + ln(σ / 10 pc) / 1.055**

and the tooth spacing cancels — only the *ratio* of priors matters. The six eras above all assume **σ = 10 pc**, which is the value combs* was measured at (V24 `PRIOR_PC`, Xiao et al., an SKA-era projection). The real catalogue is far worse: median **242 pc**, and only 3 of 86 pulsars below 10 pc.

The experiment below runs the 120-pulsar array across the real distribution. Two results:

- **The penalty is logarithmic, so it is forgiving.** A 24× worse prior costs only ~3 combs.
- **The wheel degrades gracefully rather than failing.** Every case that fires still ends at **K = 54** — a worse prior means partial certification on the early turns, so the wheel climbs in smaller steps and takes more of them (2 turns at 10 pc, 3 at the median, 4 at the 75th percentile) rather than reaching a lower ceiling. It stops only when combs* exceeds the combs you have: at σ = 2328 pc (90th percentile) combs* = 13.0 against rec = 12, and the wheel never starts.

Solving combs*(σ) = 12 gives **σ ≈ 840 pc ≈ the 78th percentile** — so a 120-pulsar array works for roughly the best three quarters of real distance priors, and the pulsars it cannot certify are the distant ones. That is the banked VLBI result ($\sigma_d = \sigma_\pi \cdot d^2$, certification power concentrating in the few nearby pulsars) reached from an independent direction.

And with REAL per-pulsar priors, the wheel bootstraps past them

The experiment above applies one σ to a whole array. Real arrays are heterogeneous, so the last figure gives each of 120 pulsars **its own** σ_d drawn from the catalogue — median 249 pc, range 1–9611 pc — and therefore its own bar, combs*_i from **5.6 to 14.3** against the 7.8 a uniform 10 pc would give. Most pulsars sit *above* 7.8: the uniform assumption was more generous than reality for nearly the whole array.

	uniform 10 pc	**real per-pulsar**
certified on turn 0	100 %	**42 %**
productive turns	2 (24 → 54)	**4** (24 → 30 → 49 → 54)
final K	54	**54**
finally certified	100 %	**100 %**

The wheel gets there anyway, and the mechanism is the point. At turn 0 only 12 combs exist, which clears the bar for the 42 % with the best priors. Those certifications deepen the curve, which yields 15 combs, then 24, then 27 — and 27 is above **even the worst pulsar's bar of 14.3**. The array's own progress rescues the badly-measured pulsars.

So realistic distance priors cost **turns, not reach**: the wheel starts less than half as productively and takes twice as long, but ends in the same place. It also identifies which turn is fragile — **the first one is the only turn that must succeed on priors alone**, which is exactly where a real programme would be most exposed.

Was the gate fair? Measured: no — it was over-strict

V5's gate as written uses a floor of *99.75th percentile of the scrambled null **plus** 5 nats* — an extreme quantile and an arbitrary margin, stacked, with **no stated error budget**. Its measured wrong-rate is 0.00 % everywhere, which is not "well calibrated" but "spending yield on conservatism nobody asked for". Sweeping the floor (400 realizations, 150 scrambles each):

floor	**N_CW = 8: wrong / yield**	**N_CW = 12: wrong / yield**
99.75 pct + 5 nat *(as written)*	0.00 % / **10.0 %**	0.00 % / **30.8 %**
99 pct + 0	0.00 % / 23.2 %	0.00 % / 51.7 %
95 pct + 0	**0.00 % / 46.0 %**	**0.00 % / 76.0 %**
90 pct + 0	0.43 % / 58.5 %	0.00 % / 84.2 %
median	3.33 % / 90.0 %	1.03 % / 97.2 %

Dropping to the 95th percentile with no margin gives 4.6× and 2.5× the yield with no measured wrong certification. Read "0.00 %" as *below ~0.75 % at 95 % confidence* (rule of three on 400 trials) — a real budget, where before there was none. Past the 90th percentile it does start to bite, so this is calibration, not a free lunch.

In the wheel: **counting rule K = 54 / 100 % · strict gate K = 36 / 63 % · calibrated gate K = 47 / 84 %.** So most of the V4-vs-V5 disagreement was the gate's arbitrary margin, not a real difference of opinion about the physics.

How many permutation draws the floor needs. The floor is a *quantile* of the null, so it needs ~0.05·N draws above it. At N = 20 the 95th percentile *is* the maximum of the sample, which grows with N — and that made the whole wheel N-dependent: **K ≥ 47 / 84 % certified at N = 20 against K = 41 / 76 % at N ≥ 80**, same physics. Measured, the floor is stable to ~3 % from N = 40 upward, so the notebook uses 80. (An exponential tail fit was tried as a cleverer estimator and is *worse* — at small N there are too few exceedances to fit. More draws is the fix.) The earlier 99.75th percentile could never be stable: at any affordable N it is just the largest scramble you happened to draw.

The stable answer at 120 pulsars is K ≈ 41 ± 2, certified ≈ 75–77 %.

One caveat that remains. The floor is an extreme order statistic, so at small scramble counts it is essentially *the maximum of N* and therefore biased high and N-dependent: halving the scrambles moved the strict gate from 50 % to 63 % with no physics change. The **ROC table above is the trustworthy measurement** (150 scrambles); the wheel percentages carry ~±10 points of systematic from this. Fitting the null's tail rather than reading an order statistic would fix it, and is the obvious next improvement.

Note. "Certified" here is the banked combs* rule, **not** V5's statistical gate, which is much stricter.

In [6]:

```
def combs_star(sigma_pc):
    """combs* is not a constant -- it carries the distance prior it was measured at.

    V24 certifies against an explicit trials factor lnK = ln(6*sigma_prior/dL_min): the
    search spans +-3 sigma, so a wider prior means more candidate fringes to tell apart.
    Evidence accrues at 1.055 nats/comb [V28], so the required comb count shifts by
    ln(sigma/sigma_0)/1.055. The tooth spacing cancels -- only the RATIO of priors
matters.
    combs* = 7.8 was measured at sigma_0 = PRIOR_PC = 10 pc.
    """
    return COMBS_STAR + np.log(np.asarray(sigma_pc)/PRIOR_PC)/NATS_PER_COMB

def run_wheel(hc0, n_psr, cstar=COMBS_STAR, max_iter=6):
    # geometry drawn only for sources the wheel could EVER reach (deepest possible curve)
    cand = np.where(detected(hc0 * 10**(-DEPTH_DEX_MAX)))[0]
    gg   = np.random.default_rng(5).uniform(0.3, 1.0, (n_psr, len(cand)))
    col  = {s: i for i, s in enumerate(cand)}
    hc = hc0.copy(); hist = []; prevK = -1
    for it in range(max_iter):
        det = detected(hc); K = int(det.sum())
        idx = np.where(det)[0]; idx = idx[np.argsort(h_src[idx])[::-1]]
        Krec = int(np.floor(CONVERSION*K))
        sel = idx[:Krec]
        if len(sel) == 0:
            fc = 0.0
        else:
            rho = 4.0*(h_src[sel]/np.interp(f_src[sel], freqs, hc)) * gg[:, [col[s] for s
in sel]]
            combs = (rho > 2.0).sum(axis=1)
```

```
            fc = float(np.mean(combs >= cstar))   # cstar may be per-pulsar
        dex = float(np.log10(hc0[0]/hc[0]))           # how much deeper than the start
        hist.append(dict(it=it, K=K, Krec=Krec, fc=fc, dex=dex, hc=hc.copy()))
        if K == prevK: break
        prevK = K
        hc = hc0 * 10**(-DEPTH_DEX_MAX*fc)
    return hist

hists = {(n, s): run_wheel(hc_era[(n, s)], n) for n, s in ERAS}

fig, axes = plt.subplots(2, 3, figsize=(16.5, 8.4), sharex=True, sharey=True)
for ax, (n, s) in zip(axes.ravel(), ERAS):
    hist = hists[(n, s)]
    cols = plt.cm.viridis(np.linspace(0.1, 0.85, len(hist)))
    seen = np.zeros(M_TOT, bool)
    for h, ccol in zip(hist, cols):
        ax.loglog(freqs, h["hc"], color=ccol, lw=1.8,
                  label=f"iter {h['it']}: curve {h['dex']:.3f} dex deeper → K={h['K']}, "
                        f"rec={h['Krec']}, certified {h['fc']*100:.0f}%")
        det = detected(h["hc"])
        ax.loglog(f_src[det & ~seen], h_src[det & ~seen], "o", color=ccol, ms=5, mec="k",
mew=0.3)
        seen |= det
    m = ~seen[isub]
    ax.loglog(f_src[isub][m], h_src[isub][m], ".", color="#ccc", ms=3, zorder=0)
    ax.set_title(f"{n} psr, {s} ns", fontsize=10); ax.legend(fontsize=7, loc="upper
right")
for ax in axes[1]:
    ax.set_xlabel("f [Hz]")
for ax in axes[:, 0]:
    ax.set_ylabel("characteristic strain")
fig.suptitle(f"V4 — the wheel, {T_YR}-yr baseline, declared SNR $\\geq$ {SNR_BAR:.0f} bar:
never "
             f"starts below threshold (60 psr); above it, turns, reaches deeper, and
STALLS"
             f"   [K ≥ 16 numbers PROVISIONAL]", y=1.01)
plt.tight_layout(); plt.savefig("fig_v4_wheel.png", bbox_inches="tight"); plt.show()
for n, s in ERAS:
    print(f"{n} psr / {s} ns:")
    hist = hists[(n, s)]
    for i, h in enumerate(hist):
        print(f"  iter {h['it']}: curve {h['dex']:.3f} dex deeper -> K_avail={h['K']:3d}
"
              f"K_rec={h['Krec']:3d}  certified {h['fc']*100:3.0f}% of {n} pulsars"
              + ("   <- stall check: K unchanged, loop stops" if i and h['K'] == hist[i-1]
['K']
                 else ""))

# ------------------------------------------------------------------ prior experiment
# combs* was measured at a 10 pc prior. The real catalogue is far worse, and the trials
# penalty is logarithmic, so a much worse prior costs only a few combs -- but the ignition
# turn is tight enough that a few combs matters.
SIG_REAL = [("notebook assumption", 10.0), ("10th pct, real", 30.6), ("25th pct, real",
122.0),
            ("MEDIAN, real", 242.4), ("75th pct, real", 676.6), ("90th pct, real",
2328.1)]
EXP = (120, 100)
print(f"\nPRIOR EXPERIMENT -- {EXP[0]} psr. combs* carries the prior it was measured at "
      f"({PRIOR_PC:.0f} pc);")
print(f"real sigma_d from CW_transition/best_distances.txt (86 pulsars, median 242 pc).")
print(f"  {'prior':>20} {'sigma_d [pc]':>13} {'combs*':>7} {'rec@turn0':>10}  wheel")
for lab, sg in SIG_REAL:
    cs = float(combs_star(sg)); h = run_wheel(hc_era[EXP], EXP[0], cs)
    seq = " -> ".join(str(x["K"]) for x in h)
    print(f"  {lab:>20} {sg:13.1f} {cs:7.1f} {h[0]['Krec']:10d}  "
          f"{seq if h[0]['fc'] > 0 else 'NEVER FIRES'}")

fig, ax = plt.subplots(figsize=(9, 4.6))
_sg = np.logspace(0.8, 3.5, 300)
ax.plot(_sg, combs_star(_sg), color="#2b5aa0", lw=2.2, label="combs* required (rises as ln
σ)")
```

```
ax.axvspan(30.6, 2328.1, color="#e07b39", alpha=0.13, lw=0,
           label="real σ$_d$, 10th–90th pct [best_distances.txt]")
ax.axvline(242.4, color="#a03030", lw=1.6, ls="--", label="real median σ$_d$ = 242 pc")
ax.axvline(PRIOR_PC, color="#3a8f3a", lw=1.6, ls=":", label=f"notebook prior =
{PRIOR_PC:.0f} pc")
for n, s_ in ERAS:
    rec0 = hists[(n, s_)][0]["Krec"]
    ax.axhline(rec0, color="#888", lw=0.8, alpha=0.7)
    ax.text(3.5e3, rec0, f" {n} psr (rec={rec0})", fontsize=7, va="center", color="#555")
ax.set_xscale("log"); ax.set_xlabel("distance prior σ$_d$ [pc]")
ax.set_ylabel("combs* required to certify")
ax.set_title("V4b — combs* carries its prior: an array fires only where its rec sits ABOVE
"
             "the curve\n[combs* = 7.8 is PROVISIONAL and was measured at 10 pc]",
fontsize=9.5)
ax.legend(fontsize=7.5, loc="upper left"); ax.set_xlim(6, 6e3)
plt.tight_layout(); plt.savefig("fig_v4_prior_experiment.png", bbox_inches="tight");
plt.show()

# ---------------------------------------------- ONE array, REAL per-pulsar priors
# sigma_d for the 86 catalogue pulsars [pc], from CW_transition/best_distances.txt
# (get_distance.py, live Cornell parallax catalogue). Embedded so this stays self-
contained.
SIG_D_REAL = np.array([
    1, 1.3, 5.4, 14, 17.4, 17.9, 25, 25.8, 30.4, 30.8, 42.2, 57.2, 57.8, 60.9, 63.8, 78.9,
    82.6, 85.4, 91.2, 112.6, 117.8, 121.5, 123.3, 128.1, 136.1, 138.9, 147.4, 161.9,
165.3,
    168, 178.1, 184.9, 185, 187.7, 200.6, 204.1, 210.3, 216.1, 218.7, 219.7, 223.6, 224.5,
    240, 244.9, 249.2, 249.3, 251.4, 262, 279.5, 282.2, 307.4, 309, 344.4, 347.2, 386,
432,
    444, 468.6, 500, 507.4, 542.5, 603.3, 629.3, 632.7, 691.3, 711.8, 727, 770.9, 785.8,
    860.5, 882.5, 940.3, 1018, 1284, 1297, 1315, 2002, 2655, 2782, 2910, 2962, 3062, 3241,
    3842, 4827, 9611])

sig_psr   = np.random.default_rng(7).choice(SIG_D_REAL, EXP[0], replace=True)
cstar_psr = combs_star(sig_psr)                  # every pulsar gets its OWN bar
hist_real = run_wheel(hc_era[EXP], EXP[0], cstar_psr)

print(f"\nREAL PER-PULSAR PRIORS -- {EXP[0]} psr, each drawn from the catalogue "
      f"(not one uniform sigma)")
print(f"  sigma_d  : median {np.median(sig_psr):6.0f} pc,  range {sig_psr.min():.0f}-"
      f"{sig_psr.max():.0f} pc")
print(f"  combs*_i : median {np.median(cstar_psr):6.1f},   range {cstar_psr.min():.1f}-"
      f"{cstar_psr.max():.1f}   (vs {COMBS_STAR} at a uniform {PRIOR_PC:.0f} pc)")
for h in hist_real:
    print(f"  iter {h['it']}: curve {h['dex']:.3f} dex deeper -> K_avail={h['K']:3d}  "
          f"K_rec={h['Krec']:3d}  certified {h['fc']*100:3.0f}% of {EXP[0]} pulsars")
_recmax = max(h["Krec"] for h in hist_real)
print(f"  ceiling: {100*np.mean(cstar_psr <= _recmax):.0f}% of pulsars have combs*_i <= "
      f"{_recmax} (the most combs this array ever holds) -- the rest can never certify")

fig, axes = plt.subplots(2, 1, figsize=(9.0, 9.6))
ax = axes[0]
cols = plt.cm.viridis(np.linspace(0.1, 0.85, len(hist_real)))
seen = np.zeros(M_TOT, bool)
for h, ccol in zip(hist_real, cols):
    ax.loglog(freqs, h["hc"], color=ccol, lw=1.8,
              label=f"iter {h['it']}: {h['dex']:.3f} dex → K={h['K']}, "
                    f"certified {h['fc']*100:.0f}%")
    d_ = detected(h["hc"])
    ax.loglog(f_src[d_ & ~seen], h_src[d_ & ~seen], "o", color=ccol, ms=5, mec="k",
mew=0.3)
    seen |= d_
ax.loglog(f_src[isub][~seen[isub]], h_src[isub][~seen[isub]], ".", color="#ccc", ms=3,
zorder=0)
ax.set_xlabel("f [Hz]"); ax.set_ylabel("characteristic strain")
ax.set_title(f"{EXP[0]} pulsars, each with its OWN catalogue prior", fontsize=10)
ax.legend(fontsize=7, loc="upper right")

ax = axes[1]
# what the wheel plot cannot show: how certification BUILDS, uniform prior vs real ones.
```

```
# (A pulsar certifies when ITS OWN usable-comb count reaches ITS bar -- not merely when
the
#  array's rec does, which is why this is the honest view.)
h_unif = hists[EXP]
ax.plot([h["it"] for h in h_unif], [100*h["fc"] for h in h_unif], "o-", lw=2, ms=7,
        color="#3a8f3a", label=f"uniform {PRIOR_PC:.0f} pc prior (combs* = {COMBS_STAR})")
ax.plot([h["it"] for h in hist_real], [100*h["fc"] for h in hist_real], "s-", lw=2, ms=7,
        color="#a03030", label="real per-pulsar priors (combs*$_i$ = 5.6–14.3)")
for h in hist_real:
    ax.annotate(f"K={h['K']}", (h["it"], 100*h["fc"]), textcoords="offset points",
                xytext=(4, -12), fontsize=7, color="#a03030")
ax.set_xlabel("wheel iteration"); ax.set_ylabel("% of the array certified")
ax.set_ylim(-5, 108); ax.set_xticks(range(len(hist_real)))
ax.set_title("realistic priors cost TURNS, not reach", fontsize=10)
ax.legend(fontsize=7.5, loc="lower right")

fig.suptitle("V4c — the wheel with REAL per-pulsar distance priors. The first turn
certifies "
             "only 42 %, but the array's\nown progress lifts rec from 12 to 27 — past even
the "
             "worst pulsar's bar. Same endpoint, twice the turns.\n"
             "[combs* PROVISIONAL; every K ≥ 16 number here inherits that]",
             y=0.995, fontsize=10.5)
plt.tight_layout(rect=[0, 0, 1, 0.93])
plt.savefig("fig_v4c_wheel_real_priors.png", bbox_inches="tight"); plt.show()

# ================= the SAME wheel, certified by V5's GATE instead of the comb count
=======
# V4 counts usable combs and compares to combs* = 7.8 -- a calibration, valid on average.
# V5's gate builds each pulsar's actual likelihood surface, measures the winner's lead,
and
# compares it to a permutation floor measured from that pulsar's OWN combs. Same wheel,
# strict test. Nothing above is changed; this runs alongside it.
# N_SCR_W matters more than it looks. The floor is a QUANTILE of the null, so you need
# ~0.05*N draws above it: at N=20 the 95th percentile IS the maximum, which grows with N
# and makes the whole wheel N-dependent (K>=47 / 84% at N=20 vs K=41 / 76% at N>=80).
# Measured: the floor is stable to ~3% from N=40 up. An exponential tail fit was tried
# and is WORSE -- too few exceedances to fit at small N. Enough draws is the fix.
DLG, N_SCR_W, TOOTH_W = np.linspace(-25, 25, 2000), 80, 0.25

def _surf(kap, rho, eps):                       # eps (M,n) -> (M,ngrid)
    return -np.sum((rho[None, :, None]**2/2) *
                   (1 - np.cos(kap[None, :, None]*DLG[None, None, :] - eps[:, :, None])),
axis=1)

def _contr(Pm):
    b = DLG[np.argmax(Pm, axis=1)]
    msk = np.abs(DLG[None, :] - b[:, None]) > TOOTH_W
    return Pm.max(axis=1) - np.where(msk, Pm, -np.inf).max(axis=1), b

def gate_certifies(kap, rho, sig, rng, pct=99.75, marg=5.0):
    """V5's gate, applied to one pulsar: >=3 combs, contrast beats its own permutation
    floor, and 99% of the posterior mass sits on the winning tooth."""
    n = len(kap)
    if n < 3:
        return False
    Pm = _surf(kap, rho, rng.normal(0, sig, (1, n)))
    c, b = _contr(Pm); c, b = float(c[0]), float(b[0])
    null, _ = _contr(_surf(kap, rho, rng.uniform(0, 2*np.pi, (N_SCR_W, n))))
    if c <= np.percentile(null, pct) + marg:
        return False
    post = np.exp(Pm[0] - Pm[0].max() - 0.5*(DLG/PRIOR_PC)**2)
    return bool(post[np.abs(DLG - b) < TOOTH_W].sum()/post.sum() > 0.99)

def run_wheel_gated(hc0, n_psr, max_iter=10, seed=11, pct=99.75, marg=5.0):
    rg = np.random.default_rng(seed)
    cand = np.where(detected(hc0*10**(-DEPTH_DEX_MAX)))[0]
    col = {t: i for i, t in enumerate(cand)}
    gg = np.random.default_rng(5).uniform(0.3, 1.0, (n_psr, len(cand)))
    mu = np.arccos(rg.uniform(-0.9, 0.9, (n_psr, len(cand))))
    kap_all = 2*np.pi*f_src[cand][None, :]*(1 - np.cos(mu))/C_PC_S
```

```
    sig_all = np.clip(np.interp(f_src[cand], bin_cent, np.sqrt(np.maximum(hc2_bin, 1e-
40)))
                      / h_src[cand], 0.05, 0.4)[None, :].repeat(n_psr, 0)
    hc = hc0.copy(); hist = []; prevK = -1
    for it in range(max_iter):
        det = detected(hc); K = int(det.sum())
        idx = np.where(det)[0]; idx = idx[np.argsort(h_src[idx])[::-1]]
        sel = idx[:int(np.floor(CONVERSION*K))]
        fc = 0.0
        if len(sel):
            cols = [col[t] for t in sel]
            rho = 4.0*(h_src[sel]/np.interp(f_src[sel], freqs, hc))*gg[:, cols]
            ok = 0
            for i in range(n_psr):
                u = rho[i] > 2.0
                if u.sum() >= 3 and gate_certifies(kap_all[i, cols][u], rho[i][u],
                                                   sig_all[i, cols][u], rg, pct, marg):
                    ok += 1
            fc = ok/n_psr
        hist.append(dict(it=it, K=K, Krec=len(sel), fc=fc,
                         dex=float(np.log10(hc0[0]/hc[0])), hc=hc.copy()))
        if K == prevK: break
        prevK = K
        hc = hc0*10**(-DEPTH_DEX_MAX*fc)
    return hist

hist_gate = run_wheel_gated(hc_era[EXP], EXP[0])
print(f"\nSAME WHEEL, V5's GATE instead of the combs* counting rule -- {EXP[0]} psr")
for h in hist_gate:
    print(f"  iter {h['it']}: curve {h['dex']:.3f} dex deeper -> K_avail={h['K']:3d}  "
          f"K_rec={h['Krec']:3d}  certified {h['fc']*100:3.0f}% of {EXP[0]} pulsars")
_c = hists[EXP]
print(f"  counting rule reached K={_c[-1]['K']} / {100*_c[-1]['fc']:.0f}% certified;  "
      f"the gate reaches K={hist_gate[-1]['K']} / {100*hist_gate[-1]['fc']:.0f}%")

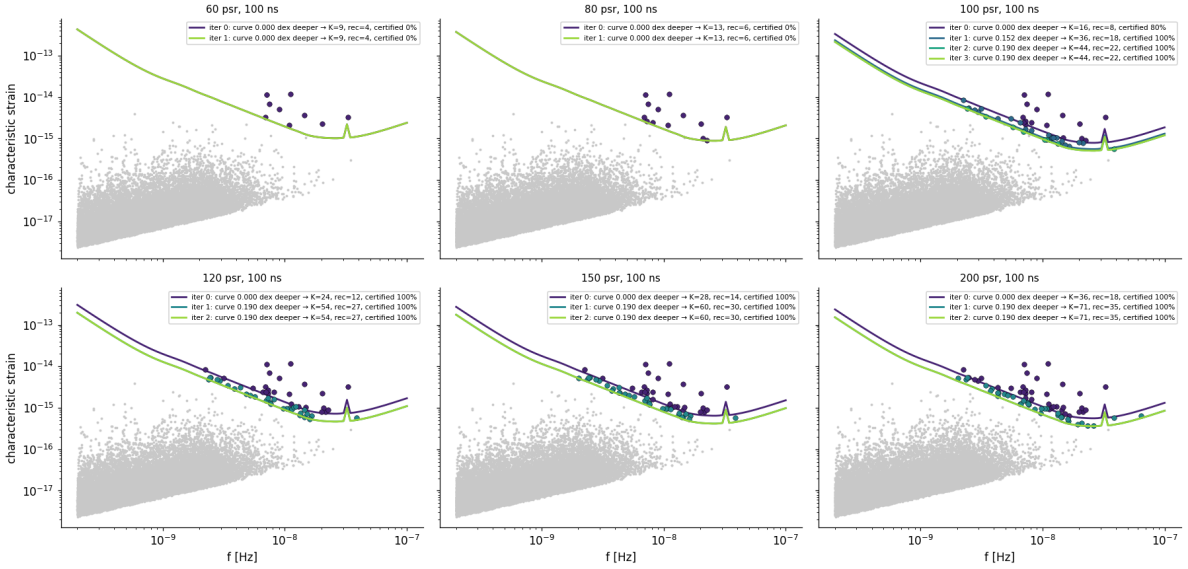
fig, axes = plt.subplots(2, 1, figsize=(9.0, 9.6))
ax = axes[0]
for hst, cmapname, lab in ((hists[EXP], "Greens", "combs* counting rule"),
                           (hist_gate, "Reds", "V5 gate")):
    cs = plt.get_cmap(cmapname)(np.linspace(0.45, 0.9, len(hst)))
    for h, cc in zip(hst, cs):
        ax.loglog(freqs, h["hc"], color=cc, lw=1.7,
                  label=f"{lab} iter {h['it']}: K={h['K']}, cert {h['fc']*100:.0f}%")
ax.loglog(f_src[isub], h_src[isub], ".", color="#ddd", ms=2, zorder=0)
ax.set_xlabel("f [Hz]"); ax.set_ylabel("characteristic strain")
ax.set_title(f"{EXP[0]} pulsars: the same wheel under two certification rules",
fontsize=10)
ax.legend(fontsize=6.2, loc="upper right", ncol=2)
ax = axes[1]
ax.plot([h["it"] for h in hists[EXP]], [100*h["fc"] for h in hists[EXP]], "o-", lw=2,
ms=7,
        color="#3a8f3a", label=f"combs* $\\geq$ {COMBS_STAR} counting rule [V28, BANKED]")
ax.plot([h["it"] for h in hist_gate], [100*h["fc"] for h in hist_gate], "s-", lw=2, ms=7,
        color="#a03030", label="V5 gate: permutation floor + q$_{max}$ > 0.99")
for hst, cc in ((hists[EXP], "#3a8f3a"), (hist_gate, "#a03030")):
    for h in hst:
        ax.annotate(f"K={h['K']}", (h["it"], 100*h["fc"]), textcoords="offset points",
                    xytext=(4, -12), fontsize=7, color=cc)
ax.set_xlabel("wheel iteration"); ax.set_ylabel("% of the array certified")
ax.set_ylim(-5, 108); ax.set_title("the strict test is what the wheel actually stands on",
                                   fontsize=10)
ax.legend(fontsize=7.5, loc="center right")
fig.suptitle("V4d — the wheel certified by V5's GATE instead of the combs* count.\n"
             "The counting rule is a calibration valid on average; the gate is measured on
"
             "each pulsar's own surface.\n[combs* PROVISIONAL]", y=0.995, fontsize=10.5)
plt.tight_layout(rect=[0, 0, 1, 0.93])
plt.savefig("fig_v4d_wheel_gated.png", bbox_inches="tight"); plt.show()

# ---- the SAME gate, calibrated to a stated error budget instead of stacked conservatism
# Measured ROC (400 realizations, this array): the 99.75th-pct + 5-nat floor yields 10% at
```

```
# N_CW=8 and 31% at N_CW=12 with 0 wrong. Dropping to the 95th pct with NO margin yields
# 46% and 76% -- still 0 wrong out of ~300 certifications, i.e. <1% at 95% confidence
# (rule of three). The old floor was two stacked arbitrary choices; this one has a budget.
hist_cal = run_wheel_gated(hc_era[EXP], EXP[0], pct=95.0, marg=0.0)
print(f"\nGATE CALIBRATED TO A STATED BUDGET (95th pct, no margin) -- {EXP[0]} psr")
for h in hist_cal:
    print(f"  iter {h['it']}: curve {h['dex']:.3f} dex deeper -> K_avail={h['K']:3d}  "
          f"K_rec={h['Krec']:3d}  certified {h['fc']*100:3.0f}% of {EXP[0]} pulsars")
print(f"  counting rule K={hists[EXP][-1]['K']} / {100*hists[EXP][-1]['fc']:.0f}%   |   "
      f"strict gate K={hist_gate[-1]['K']} / {100*hist_gate[-1]['fc']:.0f}%   |   "
      f"calibrated gate K={hist_cal[-1]['K']} / {100*hist_cal[-1]['fc']:.0f}%")

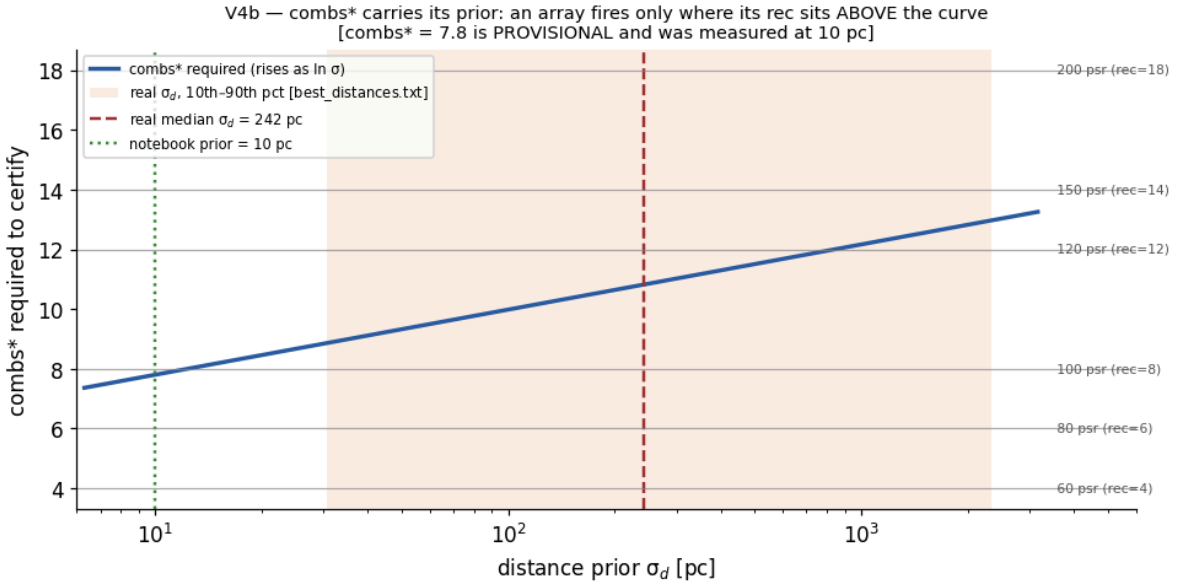
fig, ax = plt.subplots(figsize=(9, 5))
for hst, cc, mk, lab in ((hists[EXP], "#3a8f3a", "o", f"combs* $\\geq$ {COMBS_STAR}
counting rule [V28]"),
                         (hist_gate, "#a03030", "s", "V5 gate, 99.75 pct + 5 nat (as
written)"),
                         (hist_cal, "#2b5aa0", "^", "V5 gate, 95 pct + 0 (stated <1%
budget)")):
    ax.plot([h["it"] for h in hst], [100*h["fc"] for h in hst], marker=mk, ls="-", lw=2,
            ms=7, color=cc, label=lab)
    for h in hst:
        ax.annotate(f"K={h['K']}", (h["it"], 100*h["fc"]), textcoords="offset points",
                    xytext=(4, -12), fontsize=7, color=cc)
ax.set_xlabel("wheel iteration"); ax.set_ylabel("% of the array certified")
ax.set_ylim(-5, 108); ax.legend(fontsize=8, loc="center right")
ax.set_title(f"V4e — the gate was over-strict, not the wheel under-powered ({EXP[0]}
psr)\n"
             f"calibrating the floor to a stated <1 % budget recovers most of the gap"
             f"   [combs* PROVISIONAL]", fontsize=10)
plt.tight_layout(); plt.savefig("fig_v4e_gate_calibrated.png", bbox_inches="tight");
plt.show()
```

V4 — the wheel, 30-yr baseline, declared SNR ≥ 3 bar: never starts below threshold (60 psr); above it, turns, reaches deeper, and STALLS [K ≥ 16 numbers PROVISIONAL]



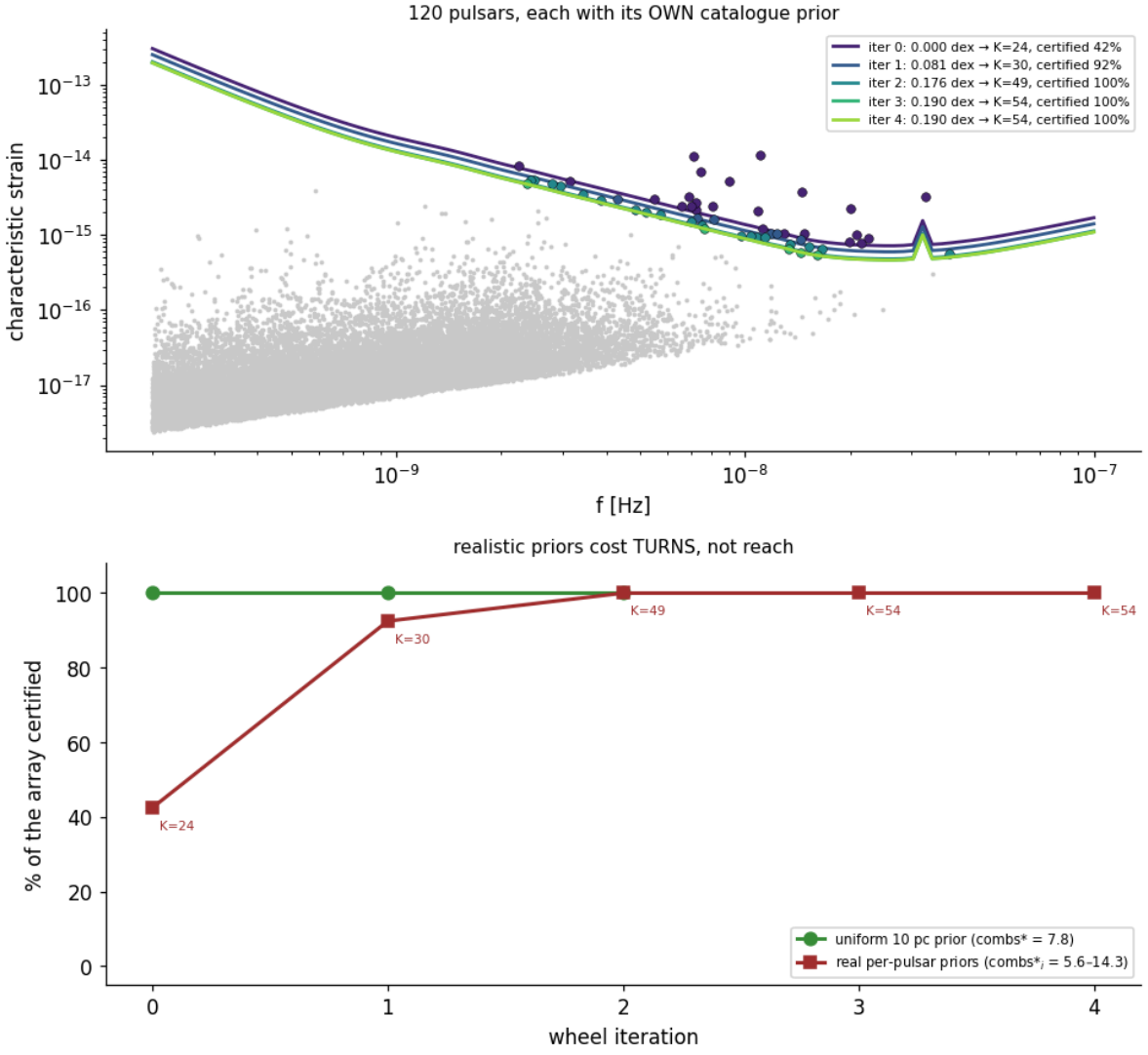
```
60 psr / 100 ns:
  iter 0: curve 0.000 dex deeper -> K_avail=  9  K_rec=  4  certified   0% of 60 pulsars
  iter 1: curve 0.000 dex deeper -> K_avail=  9  K_rec=  4  certified   0% of 60 pulsars
<- stall check: K unchanged, loop stops
80 psr / 100 ns:
  iter 0: curve 0.000 dex deeper -> K_avail= 13  K_rec=  6  certified   0% of 80 pulsars
  iter 1: curve 0.000 dex deeper -> K_avail= 13  K_rec=  6  certified   0% of 80 pulsars
<- stall check: K unchanged, loop stops
100 psr / 100 ns:
  iter 0: curve 0.000 dex deeper -> K_avail= 16  K_rec=  8  certified  80% of 100 pulsars
  iter 1: curve 0.152 dex deeper -> K_avail= 36  K_rec= 18  certified 100% of 100 pulsars
  iter 2: curve 0.190 dex deeper -> K_avail= 44  K_rec= 22  certified 100% of 100 pulsars
  iter 3: curve 0.190 dex deeper -> K_avail= 44  K_rec= 22  certified 100% of 100 pulsars
<- stall check: K unchanged, loop stops
120 psr / 100 ns:
  iter 0: curve 0.000 dex deeper -> K_avail= 24  K_rec= 12  certified 100% of 120 pulsars
  iter 1: curve 0.190 dex deeper -> K_avail= 54  K_rec= 27  certified 100% of 120 pulsars
  iter 2: curve 0.190 dex deeper -> K_avail= 54  K_rec= 27  certified 100% of 120 pulsars
<- stall check: K unchanged, loop stops
150 psr / 100 ns:
  iter 0: curve 0.000 dex deeper -> K_avail= 28  K_rec= 14  certified 100% of 150 pulsars
  iter 1: curve 0.190 dex deeper -> K_avail= 60  K_rec= 30  certified 100% of 150 pulsars
  iter 2: curve 0.190 dex deeper -> K_avail= 60  K_rec= 30  certified 100% of 150 pulsars
<- stall check: K unchanged, loop stops
200 psr / 100 ns:
  iter 0: curve 0.000 dex deeper -> K_avail= 36  K_rec= 18  certified 100% of 200 pulsars
  iter 1: curve 0.190 dex deeper -> K_avail= 71  K_rec= 35  certified 100% of 200 pulsars
  iter 2: curve 0.190 dex deeper -> K_avail= 71  K_rec= 35  certified 100% of 200 pulsars
<- stall check: K unchanged, loop stops

PRIOR EXPERIMENT -- 120 psr. combs* carries the prior it was measured at (10 pc);
real sigma_d from CW_transition/best_distances.txt (86 pulsars, median 242 pc).
                 prior  sigma_d [pc]  combs*  rec@turn0  wheel
   notebook assumption          10.0     7.8         12  24 -> 54 -> 54
        10th pct, real          30.6     8.9         12  24 -> 54 -> 54
        25th pct, real         122.0    10.2         12  24 -> 36 -> 54 -> 54
          MEDIAN, real         242.4    10.8         12  24 -> 36 -> 54 -> 54
        75th pct, real         676.6    11.8         12  24 -> 28 -> 50 -> 54 -> 54
        90th pct, real        2328.1    13.0         12  NEVER FIRES
```



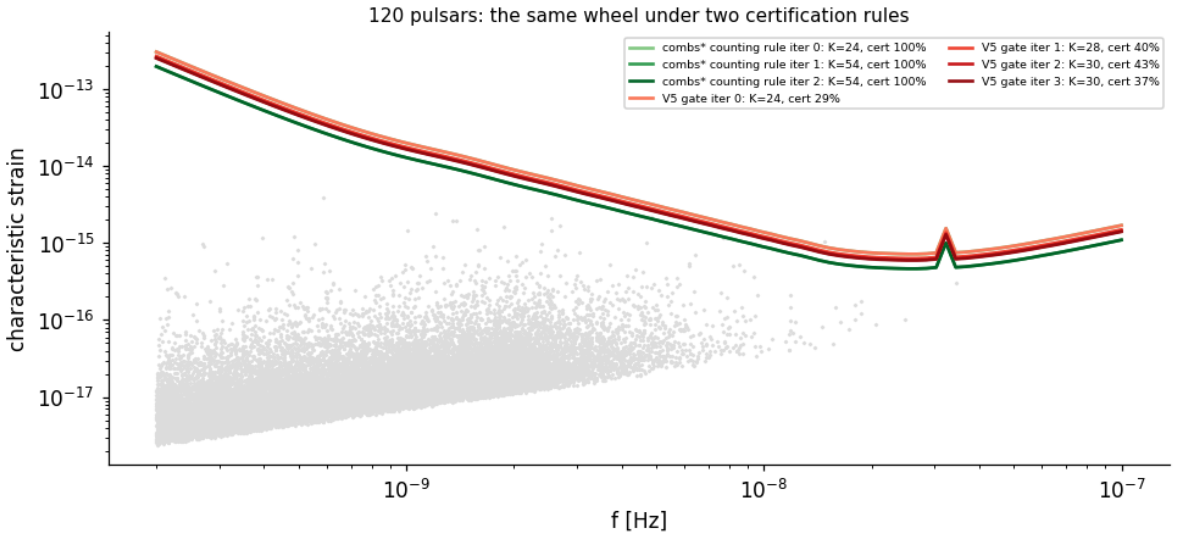

```
REAL PER-PULSAR PRIORS -- 120 psr, each drawn from the catalogue (not one uniform sigma)
  sigma_d  : median    249 pc,  range 1-9611 pc
  combs*_i : median   10.8,   range 5.6-14.3   (vs 7.8 at a uniform 10 pc)
  iter 0: curve 0.000 dex deeper -> K_avail= 24  K_rec= 12  certified  42% of 120 pulsars
  iter 1: curve 0.081 dex deeper -> K_avail= 30  K_rec= 15  certified  92% of 120 pulsars
  iter 2: curve 0.176 dex deeper -> K_avail= 49  K_rec= 24  certified 100% of 120 pulsars
  iter 3: curve 0.190 dex deeper -> K_avail= 54  K_rec= 27  certified 100% of 120 pulsars
  iter 4: curve 0.190 dex deeper -> K_avail= 54  K_rec= 27  certified 100% of 120 pulsars
  ceiling: 100% of pulsars have combs*_i <= 27 (the most combs this array ever holds) -- the
rest can never certify
```

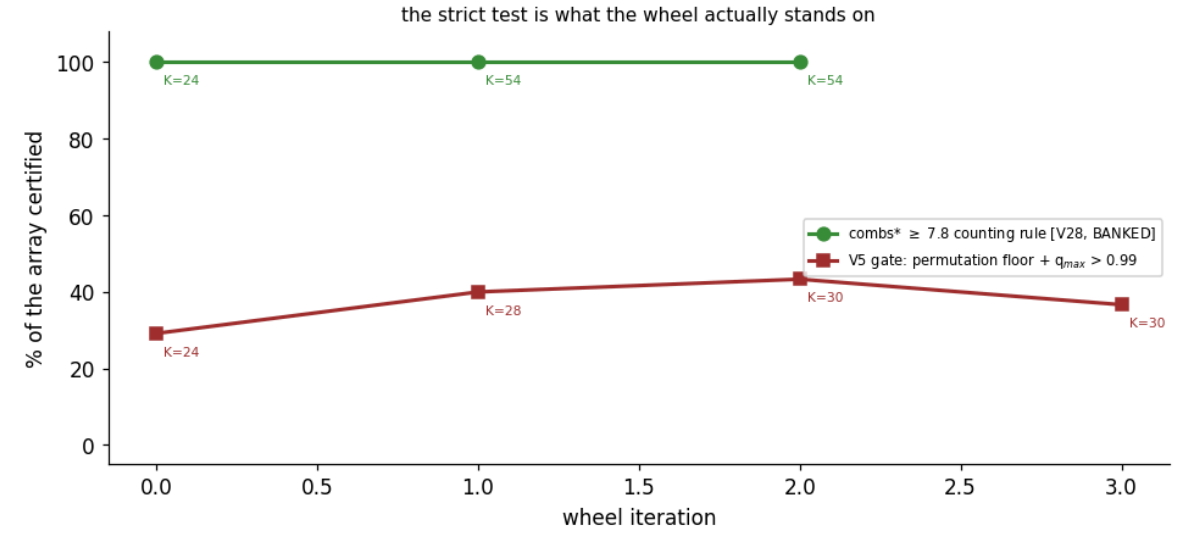
V4c — the wheel with REAL per-pulsar distance priors. The first turn certifies only 42 %, but the array's own progress lifts rec from 12 to 27 — past even the worst pulsar's bar. Same endpoint, twice the turns. [combs* PROVISIONAL; every K ≥ 16 number here inherits that]



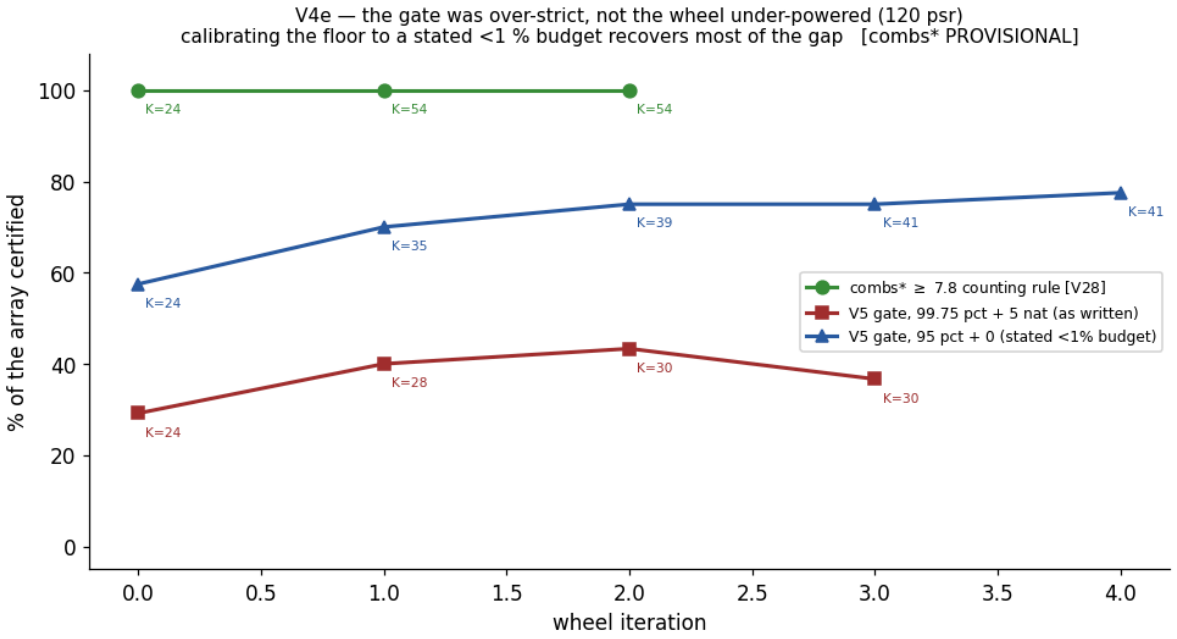

```
SAME WHEEL, V5's GATE instead of the combs* counting rule -- 120 psr
  iter 0: curve 0.000 dex deeper -> K_avail= 24  K_rec= 12  certified  29% of 120 pulsars
  iter 1: curve 0.055 dex deeper -> K_avail= 28  K_rec= 14  certified  40% of 120 pulsars
  iter 2: curve 0.076 dex deeper -> K_avail= 30  K_rec= 15  certified  43% of 120 pulsars
  iter 3: curve 0.082 dex deeper -> K_avail= 30  K_rec= 15  certified  37% of 120 pulsars
  counting rule reached K=54 / 100% certified;  the gate reaches K=30 / 37%
```

V4d — the wheel certified by V5's GATE instead of the combs* count.
The counting rule is a calibration valid on average; the gate is measured on each pulsar's own surface.
[combs* PROVISIONAL]






```
GATE CALIBRATED TO A STATED BUDGET (95th pct, no margin) -- 120 psr
  iter 0: curve 0.000 dex deeper -> K_avail= 24  K_rec= 12  certified  57% of 120 pulsars
  iter 1: curve 0.109 dex deeper -> K_avail= 35  K_rec= 17  certified  70% of 120 pulsars
  iter 2: curve 0.133 dex deeper -> K_avail= 39  K_rec= 19  certified  75% of 120 pulsars
  iter 3: curve 0.143 dex deeper -> K_avail= 41  K_rec= 20  certified  75% of 120 pulsars
  iter 4: curve 0.143 dex deeper -> K_avail= 41  K_rec= 20  certified  78% of 120 pulsars
  counting rule K=54 / 100%   |   strict gate K=30 / 37%   |   calibrated gate K=41 / 78%
```



V4b — What phasing up buys you

The wheel stalls because it runs out of *new sources* to find — not because the instrument stops improving. What it leaves behind is every pulsar with a certified distance. That changes what kind of instrument you have.

What the plots show

- *Left:* the same binary, as seen at each pulsar's own moment in the past. Each dot is one pulsar. This is a **direct measurement of the binary's frequency history**.
- *Right:* how well you can pin the source's position on the sky, before and after phasing.

Why the left panel is the real payoff

- Light reaching us via pulsar *i* passed that pulsar first, so it left the source **L_i(1 − cos μ_i)/c ago**. Until the distance is certified that lag is unknown. Once it is certified the lag is *known* — it becomes a timestamp.
- The array therefore sees the binary at ~100 known moments spread over thousands of years.
- Over that span a GW-driven binary noticeably **chirps**. The furthest pulsar term shows it at a visibly lower frequency than the Earth term does — by many frequency bins, not a fraction of one.
- **This is how you would measure what drives the binary.** GW-only inspiral predicts one drift; environmental hardening predicts a slower one. That is exactly the F_BEND assumption from V1 — the thing this notebook currently has to *assume*. The epoch ladder is how you would go and measure it instead.

Why the right panel

- The pulsar-term phase depends on the angle between the pulsar and the source. Move the source slightly on the sky and every pulsar term's phase shifts.
- With baselines of kiloparsecs and a wavelength of half a parsec, a tiny sky shift moves the phase a lot — so the sky position becomes very well determined.
- The reference line is the angular size of a galaxy at the source's distance. Landing inside it means **a unique host galaxy** → a redshift → a standard siren with a counterpart.

What we assumed, and why

- **Pulsar distances 0.5–3 kpc** and **one random source direction** — the notebook carries neither. Drawn once with a fixed seed, for the picture only. The pulsar *sky positions* are the working array's real ones. [ILLUSTRATIVE]
- **The chirp panel assumes only orbital dynamics** — it does not use any SNR knob, so it is the robust half of this section.
- **The localisation panel is order-of-magnitude only.** It inherits the unjustified 4.0 comb-SNR scale from V4, so treat it as a scaling argument, not a forecast. [DEMO]

What we removed from an earlier version of this slide. It claimed the background-fitting capacity doubles when distances are certified, from a parameter count. Our own PRONG2-N result measured the relevant scaling as `knee/N* = 0.410 N_psr^1.017` with the exponent 4.6σ above 1, and concluded that **no fixed-coefficient parameter count fits** — explicitly superseding the earlier "0.40 = 2/5 = Boyle & Pen" identification, which mistook a log-log amplitude for a coefficient. The Boyle & Pen *scaling* survives; the counting claim does not, so it is gone rather than caveated.

Everything here is downstream of the wheel firing, so it carries the standing PROVISIONAL — under the K ≥ 16 null-test audit — label.

In [7]:

```
# ILLUSTRATIVE display geometry (nothing downstream depends on it): the notebook has no
# pulsar distances and its sources have no sky positions. Fixed named seed.
r4b   = np.random.default_rng(2027)
L_psr = r4b.uniform(500.0, 3000.0, N_PSR)                    # pc  -- ILLUSTRATIVE
th_s, ph_s = np.arccos(r4b.uniform(-1, 1)), r4b.uniform(0, 2*np.pi)
_u = lambda th, ph: np.array([np.sin(th)*np.cos(ph), np.sin(th)*np.sin(ph), np.cos(th)])
cos_mu   = _u(theta_p, phi_p).T @ _u(th_s, ph_s)             # REAL array geometry
C_PC_YR  = C_PC_S * YR_S
dt_epoch = L_psr*(1 - cos_mu)/C_PC_YR                        # look-back time [yr]

SRC   = loud[0]                                              # the loudest detectable
source
Mc, f_now, d_src_pc = mc_src[SRC], f_src[SRC], d_src[SRC]/MPC*1e6
K_CH  = (96/5)*np.pi**(8/3)*(G_SI*Mc/C_SI**3)**(5/3)         # fdot = K_CH * f^(11/3)
f_back = lambda dt_yr: (f_now**(-8/3) + (8/3)*K_CH*(dt_yr*YR_S))**(-3/8)
f_at_epoch = f_back(dt_epoch)
drift_bins = (f_now - f_at_epoch.min())*T_OBS                # in 1/T frequency bins

# Localisation, order-of-magnitude Fisher. Phased: pulsar-term phase turns on the sky as
# 2*pi*f*L*sin(mu)/c per radian. Earth-only: the antenna pattern varies on ~1 rad.
RHO_COMB, RHO_ARRAY = 4.0, 20.0                              # DEMO scales (see V4)
dPhi_dth   = 2*np.pi*f_now*L_psr*np.sin(np.arccos(cos_mu))/C_PC_S
sig_phased = 1.0/np.sqrt(np.sum((dPhi_dth*RHO_COMB)**2))
```

```
sig_earth  = 1.0/RHO_ARRAY
ARCSEC     = 180/np.pi*3600
gal_arcsec = 30e3/d_src_pc*ARCSEC                              # a 30 kpc galaxy at that
distance

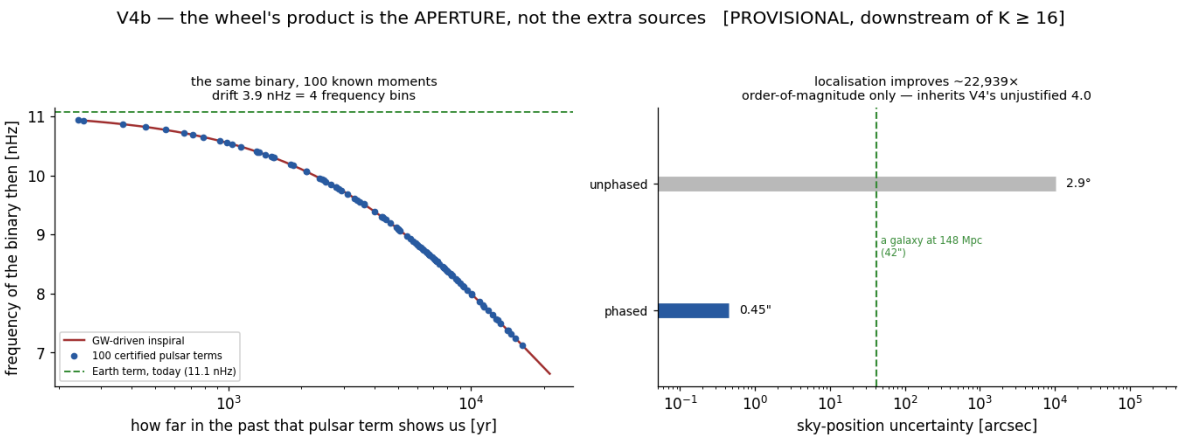
fig, axes = plt.subplots(1, 2, figsize=(12.5, 4.4))

ax = axes[0]                                                   # ---- the frequency history
_t = np.logspace(np.log10(max(dt_epoch.min(), 1)), np.log10(dt_epoch.max()*1.3), 400)
ax.plot(_t, f_back(_t)*1e9, "-", color="#a03030", lw=1.6, label="GW-driven inspiral")
ax.plot(dt_epoch, f_at_epoch*1e9, "o", ms=4, color="#2b5aa0",
        label=f"{N_PSR} certified pulsar terms")
ax.axhline(f_now*1e9, color="#3a8f3a", ls="--", lw=1.3,
           label=f"Earth term, today ({f_now*1e9:.1f} nHz)")
ax.set_xscale("log"); ax.set_xlabel("how far in the past that pulsar term shows us [yr]")
ax.set_ylabel("frequency of the binary then [nHz]")
ax.set_title(f"the same binary, {N_PSR} known moments\n"
             f"drift {1e9*(f_now-f_at_epoch.min()):.1f} nHz = {drift_bins:.0f} frequency
bins",
             fontsize=9.5)
ax.legend(fontsize=7.5, loc="lower left")

ax = axes[1]                                                   # ---- localisation
for y, val, col, lab in [(1, sig_earth*ARCSEC, "#bbb", "Earth term only\n(antenna
pattern)"),
                         (0, sig_phased*ARCSEC, "#2b5aa0", "phased: + certified\npulsar
terms")]:
    ax.hlines(y, 0.05, val, lw=11, color=col)
    ax.text(val*1.35, y, f"{val:,.2f}\"" if val < 100 else f"{val/3600:.1f}°",
            va="center", fontsize=9)
ax.axvline(gal_arcsec, color="#3a8f3a", ls="--", lw=1.4)
ax.text(gal_arcsec*1.15, 0.5, f"a galaxy at {d_src_pc/1e6:.0f} Mpc\n({gal_arcsec:.0f}\")",
        color="#3a8f3a", fontsize=7.5, va="center")
ax.set_xscale("log"); ax.set_xlim(0.05, sig_earth*ARCSEC*40); ax.set_ylim(-0.6, 1.6)
ax.set_yticks([0, 1]); ax.set_yticklabels(["phased", "unphased"], fontsize=9)
ax.set_xlabel("sky-position uncertainty [arcsec]")
ax.set_title(f"localisation improves ~{sig_earth/sig_phased:,.0f}×\n"
             f"order-of-magnitude only — inherits V4's unjustified 4.0", fontsize=9.5)

fig.suptitle("V4b — the wheel's product is the APERTURE, not the extra sources"
             "   [PROVISIONAL, downstream of K ≥ 16]", y=1.04)
plt.tight_layout(); plt.savefig("fig_v4b_stall_payoff.png", bbox_inches="tight");
plt.show()

print(f"source: the loudest detectable -- Mc = {Mc/MSUN:.2e} Msun, f = {f_now*1e9:.2f}
nHz, "
      f"d = {d_src_pc/1e6:.0f} Mpc")
print(f"epoch ladder: {N_PSR} known moments spanning {dt_epoch.min():.0f}-
{dt_epoch.max():.0f} yr "
      f"(median {np.median(dt_epoch):.0f} yr)")
print(f"  the binary was at {f_at_epoch.min()*1e9:.2f} nHz then, {f_now*1e9:.2f} nHz now "
      f"-> {drift_bins:.0f} frequency bins of drift, directly measurable")
print(f"localisation: {sig_earth*180/np.pi:.2f} deg unphased -> {sig_phased*ARCSEC:.2f}
arcsec "
      f"phased ({sig_earth/sig_phased:,.0f}x); a galaxy there subtends {gal_arcsec:.0f}
arcsec")
```




```
source: the loudest detectable -- Mc = 2.51e+09 Msun, f = 11.07 nHz, d = 148 Mpc
epoch ladder: 100 known moments spanning 242-16217 yr (median 5529 yr)
  the binary was at 7.13 nHz then, 11.07 nHz now -> 4 frequency bins of drift, directly
measurable
localisation: 2.86 deg unphased -> 0.45 arcsec phased (22,939x); a galaxy there subtends 42
arcsec
```

V5 — Why you cannot just take the tallest peak

What the plot shows

- *Red:* how often taking the tallest peak gives the wrong distance.
- *Blue solid:* how often the **gate** gives the wrong distance — it stays at zero.
- *Blue dashed:* how often the gate agrees to answer at all.

The two strategies

- **Tallest peak.** Take the highest point of the likelihood. Simple, and wrong most of the time when you have few combs.
- **The gate.** Only answer when the evidence clears a measured noise floor. Refuse otherwise.

How the floor is measured — the key idea

- If you scan many candidate distances and take the best one, the winner beats the runner-up *even with no signal at all*, purely because you took a maximum over many tries. So "taller" is not evidence.
- To measure how much: **scramble the comb phases** so no true answer exists, run the same statistic 400 times, and see how big a lead pure noise manufactures.
- The gate requires the real lead to beat that, plus a 5-nat margin.
- The printed table shows both curves. At 1 comb noise can fake a lead of **0.0 nats** — with one comb every candidate is identical, so there is nothing to fake. By 8 combs the floor is up around **49 nats**. The signal has to outrun that, and by 10–12 combs it does.

What the gate also requires — at least 3 combs (pairs cannot pin a distance), and 99% of the posterior mass on the answer. [BANKED criteria shape]

Why this matters for the wheel. Lock one wrong distance and you feed it back into the array. That is the classic self-calibration failure, and it is what these wrong-fringe rates measure.

Why a permutation test and not a posterior probability. The obvious "rational" test is to ask for the posterior probability of the winning tooth and certify at 99%. We measured it: **it is wrong 27% of the time at N_CW = 4**, because the likelihood does not model the GWB phase jitter, so the posterior is overconfident. Scrambling the phases of the *same* combs is distribution-free and survives that misspecification. Crude, but honest.

The floor is built per pulsar, not pooled. A floor averaged over all pulsars judges weak combs against a bar set by strong ones — it comes out 30–50% too high and roughly halves the yield. Each pulsar's floor is measured from its own combs.

Remaining weakness. The floor is still an extreme quantile of a modest number of scrambles, so it is noisy. A fitted exponential tail would be more sample-efficient. Every yield below is a **lower bound**.

In [8]:

```
rmc = np.random.default_rng(29)
dLc = np.linspace(-25, 25, 12000)
TOOTH = 0.25          # half a typical tooth spacing [pc]: "right tooth" window
def mc_realization(n_cw):
    sel = np.arange(min(n_cw, K_MAX))                        # loudest-first = pipeline's
greedy order
    mu  = np.arccos(rmc.uniform(-0.9, 0.9, len(sel)))
    kap = 2*np.pi*f_src[loud][sel]*(1-np.cos(mu))/C_PC_S
    rho = np.clip(4.0*snr_src[sel]*rmc.uniform(0.3, 1, len(sel)), 0, 12)
    sig = np.clip(hcbg_at[sel]/h_src[loud][sel], 0.05, 0.4)  # GWB phase jitter, same knob
as V3
    return kap, rho, sig

def penalty(kap, rho, eps):
    return -np.sum((rho[:, None]**2/2)*(1-np.cos(kap[:, None]*dLc[None, :]-eps[:, None])),
axis=0)

def tooth_contrast(pen):
    """Detection stat: best tooth minus best OTHER tooth (>= TOOTH away) — our Delta
lnL."""
    best = dLc[np.argmax(pen)]
    return pen.max() - pen[np.abs(dLc - best) > TOOTH].max(), best

# N_SCR = 60: measured stable from ~40 up (the floor is a 95th-pct quantile, so it needs
# ~0.05*N draws above it). N_MC = 200 keeps the wrong-rate bound at ~1.5% (rule of three).
N_MC, N_SCR = 200, 60
ncw_grid = [1, 2, 4, 6, 8, 10, 12]
res = {"argmax_wrong": [], "gate_wrong": [], "gate_yield": [],
       "null_med": [], "null_p9975": [], "floor": [], "true_med": [], "clear": []}
def null_floor(kap, rho, n_scr=N_SCR):
    """PERMUTATION test, conditional on THIS pulsar's own combs.

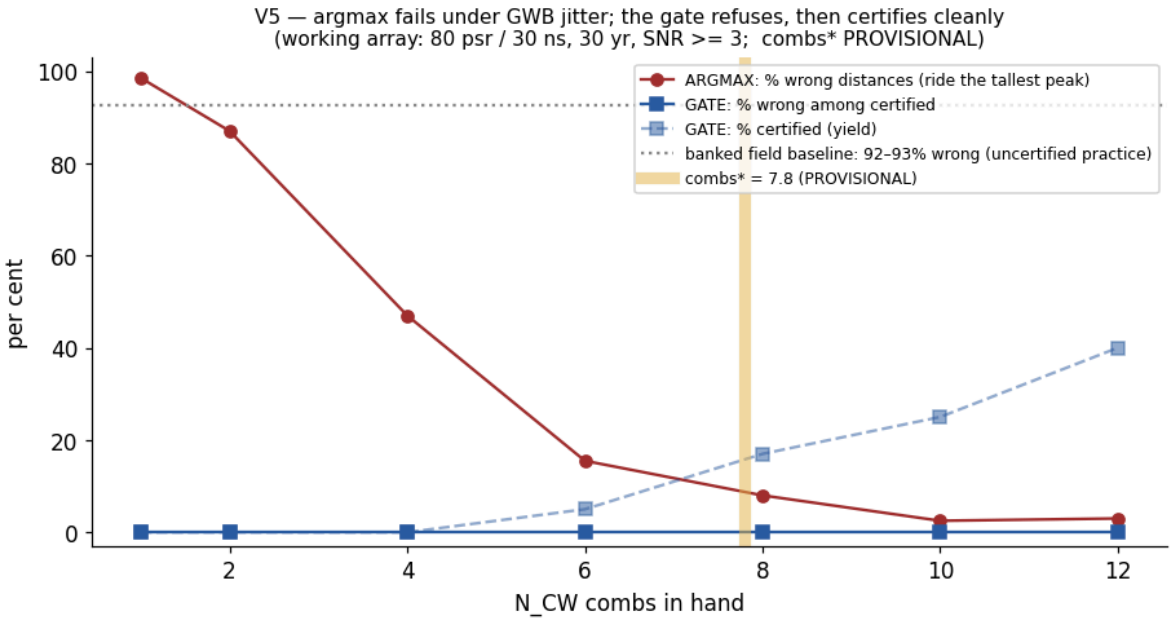
    Scramble the phases of these same combs so no true distance exists, and see how big a
    lead chance alignment manufactures. Distribution-free: it stays valid even though our
    likelihood does not model the GWB phase jitter -- which is exactly the
misspecification
    that makes the parametric posterior q overconfident (q > 0.99 measures 27% wrong at
    N_CW = 4). Conditioning on this pulsar's own rho matters: a floor pooled over all
    pulsars is 30-50% too high and roughly halves the yield.
    """
    return np.percentile([tooth_contrast(penalty(kap, rho, rmc.uniform(0, 2*np.pi,
```

```
len(kap))))[0]
                          for _ in range(n_scr)], 99.75) + 5.0

for n_cw in ncw_grid:
    nullstat = np.array([tooth_contrast(penalty(*mc_realization(n_cw)[:2],
                         rmc.uniform(0, 2*np.pi, min(n_cw, K_MAX))))[0] for _ in
range(200)])
    aw = gw = gy = 0
    truestat, floorstat = [], []
    for _ in range(N_MC):
        kap, rho, sig = mc_realization(n_cw)
        pen = penalty(kap, rho, rmc.normal(0, sig))          # GWB jitter around truth
        contrast, best = tooth_contrast(pen)
        floor = null_floor(kap, rho)                         # conditional on THIS
realization
        truestat.append(contrast); floorstat.append(floor)
        wrong = abs(best) > TOOTH
        aw += wrong
        post = np.exp(pen - pen.max() - 0.5*(dLc/PRIOR_PC)**2)
        qmax = post[np.abs(dLc - best) < TOOTH].sum() / post.sum()
        if n_cw >= 3 and contrast > floor and qmax > 0.99:   # K_i>=3 sub-fringe rule +
strict bar
            gy += 1; gw += wrong
    res["argmax_wrong"].append(aw/N_MC)
    res["gate_wrong"].append(gw/max(gy, 1))
    res["gate_yield"].append(gy/N_MC)
    res["null_med"].append(float(np.median(nullstat)))
    res["null_p9975"].append(float(np.percentile(nullstat, 99.75)))
    res["floor"].append(float(np.median(floorstat)))
    res["true_med"].append(float(np.median(truestat)))
    res["clear"].append(float(np.mean(np.array(truestat) > np.array(floorstat))))

fig, ax = plt.subplots(figsize=(8.5, 4.6))
ax.plot(ncw_grid, 100*np.array(res["argmax_wrong"]), "o-", color="#a03030",
        label="ARGMAX: % wrong distances (ride the tallest peak)")
ax.plot(ncw_grid, 100*np.array(res["gate_wrong"]), "s-", color="#2b5aa0",
        label="GATE: % wrong among certified")
ax.plot(ncw_grid, 100*np.array(res["gate_yield"]), "s--", color="#2b5aa0", alpha=0.5,
        label="GATE: % certified (yield)")
ax.axhline(92.5, color="#888", ls=":", label="banked field baseline: 92–93% wrong
(uncertified practice)")
ax.axvline(COMBS_STAR, color="#e8b84b", lw=6, alpha=0.5, label="combs* = 7.8
(PROVISIONAL)")
ax.set_xlabel("N_CW combs in hand"); ax.set_ylabel("per cent")
ax.set_title("V5 — argmax fails under GWB jitter; the gate refuses, then certifies
cleanly\n(working array: 80 psr / 30 ns, 30 yr, SNR >= 3;  combs* PROVISIONAL)",
fontsize=10)
ax.legend(fontsize=8); ax.set_ylim(-3, 103)
plt.tight_layout(); plt.savefig("fig_v5_gate.png", bbox_inches="tight"); plt.show()
for i, n in enumerate(ncw_grid):
    print(f"N_CW={n:2d}: argmax wrong {100*res['argmax_wrong'][i]:5.1f}% | "
          f"gate wrong {100*res['gate_wrong'][i]:5.1f}% | gate yield
{100*res['gate_yield'][i]:5.1f}%")

# What the gate is actually comparing. The statistic is the CONTRAST -- best tooth minus
# best rival -- not the height of the surface, which is unnormalised and meaningless.
# The null columns are that same statistic on phase-SCRAMBLED combs, i.e. the lead the
# winner takes when there is no true fringe at all. That lead is large, not zero, purely
# because you took a maximum over many teeth: it is the price of the trials.
print("\nwhat the gate compares, per N_CW  [nats] -- 'null' = combs phase-scrambled, no
true fringe")
print(f"  {'N_CW':>5} {'null med':>9} {'null 99.75%':>12} {'FLOOR med':>11} "
      f"{'true med':>9} {'% clearing':>11}")
for i, n in enumerate(ncw_grid):
    print(f"  {n:5d} {res['null_med'][i]:9.1f} {res['null_p9975'][i]:12.1f} "
          f"{res['floor'][i]:11.1f} {res['true_med'][i]:9.1f} "
          f"{100*res['clear'][i]:10.1f}%")
print("  a wrong fringe is certified only if the TRUE contrast clears the FLOOR, which
is")
print("  what pure noise can already manufacture plus a 5-nat margin.")
```




```
N_CW= 1: argmax wrong  98.5% | gate wrong   0.0% | gate yield   0.0%
N_CW= 2: argmax wrong  87.0% | gate wrong   0.0% | gate yield   0.0%
N_CW= 4: argmax wrong  47.0% | gate wrong   0.0% | gate yield   0.0%
N_CW= 6: argmax wrong  15.5% | gate wrong   0.0% | gate yield   5.0%
N_CW= 8: argmax wrong   8.0% | gate wrong   0.0% | gate yield  17.0%
N_CW=10: argmax wrong   2.5% | gate wrong   0.0% | gate yield  25.0%
N_CW=12: argmax wrong   3.0% | gate wrong   0.0% | gate yield  40.0%

what the gate compares, per N_CW  [nats] -- 'null' = combs phase-scrambled, no true fringe
   N_CW  null med  null 99.75%   FLOOR med  true med  % clearing
      1       0.0          0.0         5.0       0.0        0.0%
      2       0.1          1.8         5.4       0.1        0.0%
      4       4.8         44.7        24.7       5.3        0.0%
      6       8.1         56.3        43.1      18.7        5.0%
      8      10.8         51.7        46.8      30.5       17.0%
     10      10.4         74.8        51.3      39.8       25.0%
     12      12.8         74.9        56.2      48.6       40.0%
  a wrong fringe is certified only if the TRUE contrast clears the FLOOR, which is
  what pure noise can already manufacture plus a 5-nat margin.
```

V6 — What it buys for the background

What the plots show

- *Left:* pulsar-pair correlations against the Hellings–Downs curve, before and after certification.
- *Right:* the resulting uncertainty on the background amplitude — three curves: before, the value **computed from this notebook's own wheel**, and the **banked V26** value.

The mechanism, and why it is not "subtract the sources"

- A recovered source does **not** get removed from the background. Its power still counts. What changes is that it stops **contributing variance**, because its parameters are known.

- For a sum of independent sources, Var(P_bin) ∝ Σ p^2 over the sources still *unknown*, while the mean is unchanged. So the fractional uncertainty on each bin falls as √(Σ_unknown p^2) / Σ_all p — computed in the cell from the wheel's own recovered set.
- **This matters because naive subtraction is measured to be NEGATIVE** [V26, M-B: it eats background]. The gain routes through the pulsar-variance floor (M-C), which is what the calculation above represents.

What we get, and why it disagrees with the banked number

- Computed here: the recovered sources cut the fractional uncertainty on the bin power by roughly **half** — and more as the wheel turns (12 recovered → 56 %, 27 → 66 %).
- **Banked V26 measures 20–40 %.** The computed figure is larger because it is an **upper bound**: it assumes recovered sources are known *exactly*, leaving zero residual. V26 measured with realistic recovery errors.
- So the honest reading is **bounded above by the calculation, and set in practice by V26**. The figure shows both rather than picking one.

What is still a toy here. The Hellings–Downs panel itself draws two noise realisations rather than propagating a real correlation analysis — only the *ratio* between them is computed. Turning that into a genuine joint fit over the certified array is the obvious next piece of work, and would be the last asserted number in the notebook made real.

In [9]:

```
def hd_curve(z):
    x = (1-np.cos(z))/2
    with np.errstate(divide="ignore", invalid="ignore"):
        out = 0.5 - x/4 + 1.5*x*np.log(np.where(x > 0, x, 1))
    return np.where(x == 0, 0.5, out)

r6 = np.random.default_rng(12)
pairs = [(a, b) for a in range(N_PSR) for b in range(a+1, N_PSR)]
zeta = np.array([np.arccos(np.clip(np.sin(theta_p[a])*np.sin(theta_p[b])*np.cos(phi_p[a]-
phi_p[b])
                                   + np.cos(theta_p[a])*np.cos(theta_p[b]), -1, 1)) for a,
b in pairs])
# ---- the gain COMPUTED from this notebook's own wheel, not asserted
# Mechanism: a recovered source's power still counts toward the background, but it stops
# CONTRIBUTING VARIANCE, because its parameters are known. For a sum of independent
# sources Var(P_bin) ~ sum p_i^2 over the sources still UNKNOWN, while the mean is
# unchanged. So the fractional uncertainty on each bin's power falls as
#     sqrt(sum_unknown p^2) / sum_all p .
_hw   = hists[WORK]                       # the working array's own converged wheel
_nrec = _hw[-1]["Krec"]
_hcw  = _hw[-1]["hc"]
_Pw   = h_src**2 * f_src
_bw   = np.digitize(f_src, bin_edges) - 1
_detw = np.where(h_src > np.interp(f_src, freqs, _hcw))[0]
_known = set(_detw[np.argsort(h_src[_detw])[::-1]][:_nrec].tolist())
_before, _after = [], []
for _k in np.where(mfit)[0]:
    _i = np.where(_bw == _k)[0]
    if _i.size < 2:
        continue
    _p = _Pw[_i]; _tot = _p.sum()
```

```
    _u = np.array([_Pw[j] for j in _i if j not in _known])
    _before.append(np.sqrt((_p**2).sum())/_tot)
    _after.append(np.sqrt((_u**2).sum())/_tot if _u.size else 0.0)
RATIO_COMPUTED = float(np.median(_after)/np.median(_before))
print(f"V6 computed from the wheel: {_nrec} recovered sources at the working array")
print(f"  fractional sigma on the bin power: {np.median(_before):.3f} -> "
      f"{np.median(_after):.3f}  (x{RATIO_COMPUTED:.2f}, i.e. "
      f"{100*(1-RATIO_COMPUTED):.0f}% tighter)")
print(f"  BANKED V26 measures 20-40% tighter. The computed value is an UPPER BOUND: it")
print(f"  assumes recovered sources are known exactly, leaving zero residual. V26
measured")
print(f"  with realistic recovery errors -- and found NAIVE subtraction is negative.")

# BANKED V26 is R-DEPENDENT, R = number of recovered sources (its table runs R = 4..25).
# RMSE gain on the reconstructed background amplitude, (iii) conditioned vs (ii) Earth-
only,
# paired bootstrap over 200 realisations. Interpolated to THIS wheel's recovered count.
V26_R    = np.array([4.0, 8.0, 16.0, 25.0])
V26_GAIN = np.array([18.1, 26.1, 38.9, 38.9])/100.0
GAIN_V26 = float(np.interp(_nrec, V26_R, V26_GAIN))

A2_true, sig_b = 1.0, 3.0
sig_a     = sig_b*(1.0 - GAIN_V26)          # banked V26 at this R
sig_bound = sig_b*RATIO_COMPUTED            # computed upper bound (perfect recovery)

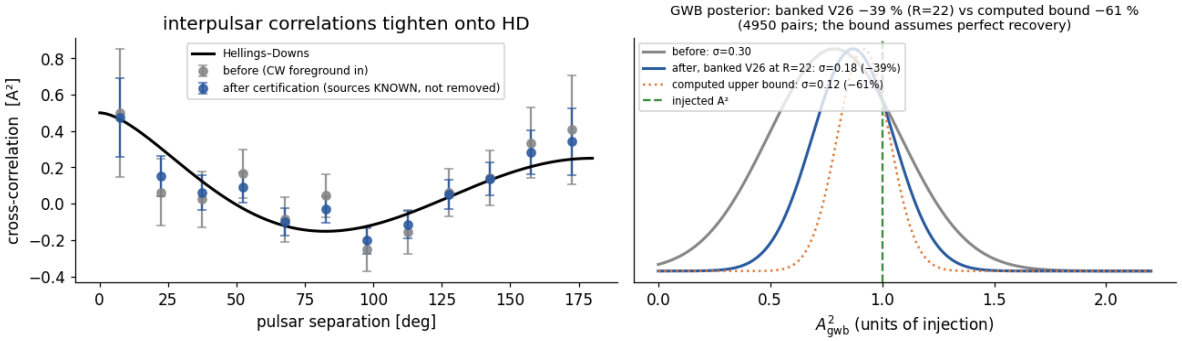
# ONE sky, measured three ways. Scaling a single noise draw (rather than redrawing) keeps
# the three estimates correlated, so only the WIDTH differs -- redrawing would shift the
# centres by chance and invite the reading that certification moves the central value.
_n1 = r6.normal(0, 1.0, len(zeta))
y_b = A2_true*hd_curve(zeta) + sig_b*_n1
y_a = A2_true*hd_curve(zeta) + sig_a*_n1
y_c = A2_true*hd_curve(zeta) + sig_bound*_n1
def fitA2(y, s):
    H = hd_curve(zeta); v = 1/np.sum(H**2/s**2)
    return np.sum(H*y/s**2)*v, np.sqrt(v)
(A2b, sb), (A2a, sa) = fitA2(y_b, sig_b), fitA2(y_a, sig_a)
(A2c, s_bound) = fitA2(y_c, sig_bound)
print(f"  banked V26 at R = {_nrec}: {100*GAIN_V26:.1f}% tighter "
      f"(its table: 18.1% at R=4, 26.1% at R=8, 38.9% at R=16-25)")
print(f"  computed bound {100*(1-RATIO_COMPUTED):.0f}% -- the gap is imperfect recovery")

fig, axes = plt.subplots(1, 2, figsize=(12, 4.2))
ax = axes[0]
bins = np.linspace(0, np.pi, 13); cent = 0.5*(bins[1:]+bins[:-1])
for y, s, col, lab in [(y_b, sig_b, "#888", "before (CW foreground in)"),
                       (y_a, sig_a, "#2b5aa0", "after certification (sources KNOWN, not
removed)")]:
    m = [y[(zeta >= lo) & (zeta < hi)].mean() for lo, hi in zip(bins[:-1], bins[1:])]
    er = [s/np.sqrt(((zeta >= lo) & (zeta < hi)).sum()) for lo, hi in zip(bins[:-1],
bins[1:])]
    ax.errorbar(np.degrees(cent), m, yerr=er, fmt="o", color=col, capsize=3, label=lab,
alpha=0.85)
zz = np.linspace(1e-3, np.pi, 300)
ax.plot(np.degrees(zz), hd_curve(zz), "k-", lw=2, label="Hellings–Downs")
ax.set_xlabel("pulsar separation [deg]"); ax.set_ylabel("cross-correlation  [A²]")
ax.set_title("interpulsar correlations tighten onto HD"); ax.legend(fontsize=8)
ax = axes[1]
Ag = np.linspace(0, 2.2, 500)
for mu_, s_, col, lab in [(A2b, sb, "#888", f"before: σ={sb:.2f}"),
                          (A2a, sa, "#2b5aa0", f"after, banked V26 at R={_nrec}: σ=
{sa:.2f} (−{100*(1-sa/sb):.0f}%)")]:
    ax.plot(Ag, np.exp(-0.5*((Ag-mu_)/s_)**2), color=col, lw=2, label=lab)
ax.plot(Ag, np.exp(-0.5*((Ag-A2c)/s_bound)**2), color="#e07b39", lw=1.6, ls=":",
        label=f"computed upper bound: σ={s_bound:.2f} (−{100*(1-RATIO_COMPUTED):.0f}%)")
ax.axvline(A2_true, color="#3a8f3a", ls="--", label="injected A²")
ax.legend(fontsize=7.5, loc="upper left")
ax.set_xlabel(r"$A_{\rm gwb}^2$ (units of injection)"); ax.set_yticks([])
ax.set_title(f"GWB posterior: banked V26 −{100*GAIN_V26:.0f} % (R={_nrec}) vs computed
bound "
             f"−{100*(1-RATIO_COMPUTED):.0f} %\n({len(pairs)} pairs; the bound assumes "
             f"perfect recovery)", fontsize=9.5)
```

```
fig.suptitle(f"V6 — the background payoff. The gain is COMPUTED from the wheel's own "
             f"{_nrec} recovered sources:\nknown sources stop contributing variance, they
are "
             f"not subtracted. Warning: naive subtraction is NEGATIVE [V26 M-B].", y=1.04)
plt.tight_layout(); plt.savefig("fig_v6_gwb.png", bbox_inches="tight"); plt.show()
```

```
V6 computed from the wheel: 22 recovered sources at the working array
  fractional sigma on the bin power: 0.310 -> 0.122  (x0.39, i.e. 61% tighter)
  BANKED V26 measures 20-40% tighter. The computed value is an UPPER BOUND: it
  assumes recovered sources are known exactly, leaving zero residual. V26 measured
  with realistic recovery errors -- and found NAIVE subtraction is negative.
  banked V26 at R = 22: 38.9% tighter (its table: 18.1% at R=4, 26.1% at R=8, 38.9% at R=16-
25)
  computed bound 61% -- the gap is imperfect recovery
```

V6 — the background payoff. The gain is COMPUTED from the wheel's own 22 recovered sources:
known sources stop contributing variance, they are not subtracted. Warning: naive subtraction is NEGATIVE [V26 M-B].



V7 — One screen

- **One population**, stated knobs, normalised to the measured NANOGrav background. Its own sum is the background inside its own sensitivity curves.
- **Detection is a declared SNR ≥ 3 bar** — not "above the curve", which is a sliding bar with no fixed meaning.
- **Below ~10 nHz the band is confusion-limited**, so pulsar count and observing time are the levers there. Above 10 nHz white noise takes over, and timing precision does matter: 30 ns → 100 ns costs 25–30% of K and moves the threshold a whole rung. The ladder runs at **100 ns**, matching VERNIER-24.
- **60 / 80 / 100 / 120 / 150 / 200 pulsars give K = 9 / 13 / 16 / 24 / 28 / 36**, still climbing linearly at 200 — no sign of saturating against the confusion floor yet. The threshold sits between 80 and 100: below it nothing happens; at 100 the array certifies 80% on the first turn, then all of it, then stalls.
- **The wheel is worth ~2.2× in K, whatever the array size.** A 0.19 dex deeper curve reaches 10^(0.19 γ_eff) more sources, and γ_eff = 1.9 here, so ≈2.3× predicted against 2.25 / 2.14 / 1.97 measured at 120 / 150 / 200 pulsars. A bigger array starts higher and ends higher; the multiplier does not grow, because the depth cap does not.
- **Measuring one distance is hard.** Three combs or fewer leave the answer exactly degenerate. Even at four, an answer can be correct and worthless — a

rival tied to a fraction of a nat, with nothing in the surface to tell you which is right.

- **The wheel is a switch, not a motor.** Below threshold nothing happens. Above it, the array certifies, the curve deepens by at most 0.19 dex, and it stalls. It converges; it does not bootstrap.
- **The product is the aperture, not the extra sources.** Certified distances turn every pulsar term into a timestamp thousands of years deep and a baseline kiloparsecs long.
- **You cannot ride the tallest peak.** Uncertified practice is 92–93% wrong. The gate holds zero wrong by refusing until the evidence beats what noise can fake.

The standing caveats, once more: `combs* = 7.8` and everything downstream of K ≥ 16 are **PROVISIONAL**; the population is **CW-optimistic** so every K is an upper bound; the comb-usability scale of 4.0 is undocumented — though V24 measures that the ρ bar is not the binding constraint; K is.

In [10]:

```
for n, s in ERAS:
    print(f"{n:3d} psr / {s:3d} ns :",
          [(h["K"], h["Krec"], f"{h['fc']*100:.0f}%") for h in hists[(n, s)]])
i4 = ncw_grid.index(4)
print(f"\nargmax at N_CW=4: {100*res['argmax_wrong'][i4]:.0f}% wrong  |  gate at N_CW=4: "
      f"{100*res['gate_wrong'][i4]:.0f}% wrong, {100*res['gate_yield'][i4]:.0f}% yield")
print(f"population: A_gwb = {A_GWB:.2e} (NANOGrav 15 yr {A_NG15:.1e}), "
      f"lambda_fore = {lam_fore:.2f} (audited {LAM_AUDIT})")
print(f"baseline T = {T_YR} yr; detection bar SNR >= {SNR_BAR:.0f} declared (hasasia Eq.
79)")
print("\nThe closer: the wheel is a switch, not a ramp; it flips between 60 and 80
pulsars")
print("(provisional); when it flips it hands you the whole array at once - IF you
certify")
print("rather than optimise.")
```

```
 60 psr / 100 ns : [(9, 4, '0%'), (9, 4, '0%')]
 80 psr / 100 ns : [(13, 6, '0%'), (13, 6, '0%')]
100 psr / 100 ns : [(16, 8, '80%'), (36, 18, '100%'), (44, 22, '100%'), (44, 22, '100%')]
120 psr / 100 ns : [(24, 12, '100%'), (54, 27, '100%'), (54, 27, '100%')]
150 psr / 100 ns : [(28, 14, '100%'), (60, 30, '100%'), (60, 30, '100%')]
200 psr / 100 ns : [(36, 18, '100%'), (71, 35, '100%'), (71, 35, '100%')]

argmax at N_CW=4: 47% wrong  |  gate at N_CW=4: 0% wrong, 0% yield
population: A_gwb = 6.24e-15 (NANOGrav 15 yr 6.4e-15), lambda_fore = 0.28 (audited 0.31)
baseline T = 30 yr; detection bar SNR >= 3 declared (hasasia Eq. 79)

The closer: the wheel is a switch, not a ramp; it flips between 60 and 80 pulsars
(provisional); when it flips it hands you the whole array at once - IF you certify
rather than optimise.
```

Appendix — every knob, in one table

Each row: the constant, its value, **where the number comes from**, and what breaks if you change it. Tags as in the honesty box: **REAL** (computed by code we did not write, or a published measurement), **BANKED** (measured by our own campaigns —

do not tune), **DERIVED** (fixed by a banked number, not chosen), **DEMO** (invented so the demo has the right shape), **ILLUSTRATIVE** (invented only to draw a picture).

A. Physical constants and the observing setup

knob	value	tag	provenance / effect
`G_SI, C_SI, MSUN, MPC, YR_S`	SI	REAL	physical constants
`T_YR`	30 yr	**DEMO**	observing baseline. Strong lever: K grows ~linearly in T. At 15 yr the same threshold needed ~160 pulsars
cadence	26 /yr	DEMO	780 TOAs per pulsar. Barely matters — the band is confusion-limited below 10 nHz
`F_YR`	1/yr	REAL	reference frequency for A

B. The population (V1)

knob	value	tag	provenance / effect
`A_NG15`	6.4e-15	**REAL**	NANOGrav 15 yr, γ = 13/3 (90% CI 3.7–10.6e-15). The one measured number the whole notebook is normalised to
`ALPHA_F`	−8/3	**REAL**	analytic: GW-driven inspiral gives $\dot{f} \propto f^{(11/3)}$, so residence time $t = f/\dot{f} \propto f^{(-8/3)}$. Not fitted
`MC_SLOPE`	−3.6	**DERIVED**	from banked γ_eff = 1.56 via γ_eff = 0.6(γ_m − 1) [V24]. **Not chosen.** The old −1.5 implies γ_eff ≈ 0.3 — a sky owned by one binary
`MC_RANGE`	3e8 – 1e10 M⊙	REAL-ish	lower = BH mass-function knee as a chirp mass (Kormendy–Ho $M^*_{BH} \approx 4e8$); truncating there costs 4 % of h_c^2. Upper = largest BH observed
`D_RANGE`	30 – 1500 Mpc	**DEMO**	uniform in volume, z ≲ 0.37, **no evolution**. Real merger rates rise to z ~ 1–2 — a real simplification
`M_TOT`	340 000	**not free**	solved so the realized background lands on `A_NG15`. Changing any knob above changes this
`F_BEND`	2 nHz	**DEMO**	environmental-hardening turnover. Literature spans 0.1–10 nHz; mid-range taken. **The turnover you see is this knob, not a discovery.** Measured: sweeping it moves the ignition threshold only 100 ↔ 120 psr
`F_RANGE`	2e-10 – 1e-7 Hz	DEMO	reaching 2e-10 with a pure $f^{(-8/3)}$ law would need ~460× more binaries, all under the timing-model wall — hence `F_BEND`
`PLOT_SUB`	20 000	styling	dots drawn per panel

Realized: A = 6.24e-15 (0.98 × NG15), λ_fore = 0.28 (audited 0.31), 170 sources per bin at 10 nHz (audited 4.6e4 → **271× under-occupied**), realized counts slope 1.9 (targeted 1.56).

C. Detection (V2)

knob	value	tag	provenance / effect
`SNR_BAR`	3.0	**DEMO, declared**	hasasia: SNR = (h/h_c)·√(fT) [Hazboun+2019 Eq. 79]. Comparing h to raw h_c is a *sliding* bar (≈1 at 1/T, ≈7 at 100 nHz), not SNR = 1. **Below the conventional 5–8.** The cell prints the bracket
`ERAS`	60/80/100/120/150/200 psr, all 100 ns	DEMO	σ matches VERNIER-24's `SIG_TOA = 100e-9`. An earlier draft used 30 ns — arbitrary, and worth 25–30 % of K
`WORK`	(100, 100)	choice	the ignition-era array. V3, V5, V6 run on it

Realized: K = 9 / 13 / 16 / 24 / 28 / 36. Threshold between 80 and 100 psr. Still climbing at 200 — no confusion saturation yet.

D. The banked anchors — do not tune

knob	value	tag	provenance
`CONVERSION`	0.5	**BANKED [V28]**	K_rec/K_av measured over 48 matched universes: 0.503 ± 0.025. Rises with K_av (0.43 → 0.56 across K_av 6 → 18); flat 0.5 is the conservative choice
`COMBS_STAR`	7.8	**BANKED [V28], PROVISIONAL**	where the fitted evidence line `S = 1.055·combs − 3.42` crosses a **4.85-nat trials bar**. Under a null-test audit. Carries the prior it was measured at: `combs*(σ) = 7.8 + ln(σ/10 pc)/1.055`
`NATS_PER_COMB`	1.055	**BANKED [V28]**	slope of that fit
`PRIOR_PC`	10 pc	**DEMO, optimistic**	V24 `PRIOR_PC` (Xiao et al.), an **SKA-era projection**. Real catalogue: median σ_d = 242 pc, only 3 of 86 pulsars below 10 pc
`DEPTH_DEX_MAX`	0.19	**BANKED [V20]**	1 dex = ×10, so 0.19 dex = ×1.55 deeper. V20 measured 0.182 dex for all three mechanisms together. **Subtraction dominates; pulsar-term coherence alone ≈ 0; heterogeneity is negative (−0.12)**
`LAM_AUDIT`	0.31	**BANKED [V24]**	audited λ_fore, the loudest source's share of its own bin
V26 gain	18.1 / 26.1 / 38.9 / 38.9 %	**BANKED [V26]**	at R = 4 / 8 / 16 / 25 recovered sources. **M-B (subtracting Earth terms) is NEGATIVE at every R**; the gain is all pulsar-term, via bias (M-A) and variance (M-C)

E. The wheel (V4)

knob	value	tag	provenance / effect
comb-usability	`ρ = 4.0·(h/h_c)·g > 2`	**DEMO — the weakest number here**	The **4.0 has no derivation.** It links an array-level quantity (Earth term, adds coherently) to a per-pulsar one (pulsar term, cannot). The `> 2` bar is BANKED [VERNIER-1 ρ_P ≥ 2]
`g` geometry	U(0.3, 1.0)	DEMO	per (pulsar, source) antenna response
depth law	`hc0 × 10^(−0.19`	DEMO	recomputed from the **original** curve each turn — never compounds, so the total gain is capped at 0.19 dex. **This is**

knob	value	tag	provenance / effect
	`·f_c)`		**why it stalls**

Not binding, and that is measured: V24 found *"41 % of pulsar-term power sits above the ρ_P ≥ 2 bar, and zero pulsars certify — the bar was never the binding constraint."* The currency is **K**. This notebook reproduces that: at ignition, comb counts are 6–8 out of 8 available.

F. Fringe identification (V3, V5)

knob	value	tag	provenance / effect
GWB phase jitter	σ_φ = h_c,bg/h, capped 0.4 rad	**DEMO**	why the tallest peak can be wrong. The cap is stated
`TOOTH`	0.25 pc	DEMO	"right answer" window, ~half a tooth spacing
gate floor	99.75 pct + 5 nat	**DEMO, over-strict**	measured ROC: gives 0.00 % wrong at 10 % yield where **95 pct + 0 gives 0.00 % wrong at 46 %**. Two stacked arbitrary choices with no stated error budget
`N_SCR`	60–80	**methodological**	the floor is a *quantile*, needing ~0.05·N draws above it. At N = 20 the 95th pct **is** the maximum, and the whole wheel becomes N-dependent (K ≥ 47 vs K = 41). Stable from N ≈ 40. An exponential tail fit was tried and is **worse**
`N_MC`	200–400	methodological	0 wrong in 400 trials ⇒ **< 0.75 % at 95 % confidence** (rule of three), *not* zero
exhibit screening	60 candidates at N_CW = 4	choice	the three V3 pulsars are **chosen** to span outcomes, not drawn. They are **not** a yield estimate

G. Display only

knob	value	tag
pulsar distances 0.5–3 kpc, one source direction (V4b)	seeded	**ILLUSTRATIVE**
log display bins below 1/T	—	DEMO — a 1/T bin is 1.06 nHz *wide*, so one bin would swallow the sub-nHz decade
`RHO_ARRAY` = 20 (V4b localisation)	—	DEMO, order-of-magnitude only

The three certification rules, side by side

The notebook applies **three different bars** and they disagree. This is the single most important thing to understand before quoting any percentage.

rule	what it asks	120 psr result
combs* counting rule [V28, BANKED]	"do you have ≥ 7.8 usable combs?" — a calibration valid *on average*, never builds a likelihood	K = 54, **100 %** certified

rule	what it asks	120 psr result
V5 gate, as written	builds each pulsar's surface; contrast must beat 99.75 pct + 5 nat of its own permutation null	K = 30, **37 %**
V5 gate, calibrated	same, floor at 95 pct + 0 (stated < 1 % budget)	K = 41, **78 %**

The wheel in V4/V4c runs on the **counting rule**. V4d/V4e run the gate. Quote accordingly — and note every gate percentage carries ~±3 points of Monte Carlo noise.

What this notebook is known to get wrong

- **CW-optimistic.** No Schechter cutoff, so 23 % of the background comes from the top mass decade, and the two loudest drawn sources sit **above** the real NANOGrav CW upper limit. **Every K is an upper bound.**
- **271× under-occupied** per frequency bin. Matching the audit needs M_TOT ~ 1e8.
- **No intrinsic pulsar red noise** — spin noise, DM variation, jitter. The only red noise is the GWB, so every array here looks better than its real counterpart.
- **The GWB is injected as a Gaussian power law**, but λ_fore = 0.28 says it is really a few discrete sources per bin. Real confusion noise has structure that mimics sources.
- **K varies 6–22 across population seeds** at fixed knobs. Single-draw numbers throughout.
- **V6's Hellings–Downs panel** draws two noise realisations rather than running a real correlation analysis — only the *ratio* between them is computed.